

ET-SoC-1 Errata

This document is for internal use only

This document contains all known errata in A0 Silicon

Table of Contents

Revision History	5
1. Minion and Minion Shire	6
1.1. Denial of service in broadcast widget	6
1.2. TensorLoadSetupB does not support context switch	7
1.3. Gather/Scatter instruction is missing some load/store operations since gsc_progress register is not updated	8
1.4. Field CacheOp_Max in register ucache_control cannot be set to 0	9
1.5. HART ESR address is not correctly checked	10
1.6. Enabling virtual memory can lead to undefined behavior	11
1.7. When the "stride" value is set unrealistically high, address overflow may not be detected	12
1.8. The mask value of contiguous vector stores that bypass L1 is ignored	13
1.9. Using message port extensions can lead to undefined behavior	14
1.10. Tensor Conv clashes with a write to tensor_mask	15
1.11. Starting a TensorQuant operation may incorrectly update bit 4 (SCP not enabled) of the Tensor Error CSR for threads 0 and 1	16
1.12. Branch Taken events for performance counters are not correctly generated	17
1.13. Minion data breakpoints not working properly on CacheOps and TensorLoads/Stores	18
1.14. VPU not connected to Minion Shire warm reset	19
1.15. Part of a misaligned access may execute with the wrong privilege	20
1.16. Exceptions on misaligned accesses may not release Minion resources	21
1.17. The SBM may drop ESR write responses if there is a broadcast inflight and BPAM accesses to the same endpoint	22
1.18. Resume state machine only works for Minion 0	23
1.19. Hart can execute unexpected instructions in debug mode	25
1.20. Bad mtval after debug exception for PC that has fetch AF in the second half of the instruction	26
1.21. Amocmpswap x31 implicit dependency not correctly tracked	27
1.22. Writes to PMU registers may happen twice	28
1.23. Simultaneous reading from both Minion threads of the PMU registers may return incorrect data to the second thread doing the read	29
1.24. Debug access to the L1 ICache tag memory incorrectly enables simultaneous access to the L0 microcache 0 memories	30
1.25. Write to the Tensor mask does not wait for previous tensor_conv_size	31
1.26. For CacheOps, the Tensor mask for one thread can leak and be used as the Tensor mask for the other thread	32
1.27. AndOrTreeL2 'allhalted' and 'allrunning' fields are incorrect when there are no harts selected	33
1.28. Intensive usage of UC operations in one Minion thread may produce starvation to the other thread	34
1.29. Minion VPURF Timing Path	35

1.29.1. Workaround for Type A	36
1.29.2. Workaround for Type B	37
1.29.3. Workaround for Type C	38
1.29.4. Workaround for Type D	38
1.29.5. Workaround for Type E	39
1.29.6. Workaround for Type F	39
1.29.7. Workaround for Type G	40
1.30. Reset assertion may induce logic errors if not all the resets are asserted	42
2. Maxion and IOShire	43
2.1. Accessing a certain undefined address space (0x10) may hang the SP	43
2.2. Maxion return event counters miss counting some returns	44
2.3. Maxion PTW requests are not checked for access violations	45
2.4. Accesses to address-map holes in the Maxion subsystem result in undefined behavior	46
2.5. Maxion may raise speculative interrupts for bus errors on data accesses	47
2.6. Maxion debug single step functionality requires serial execution mode	48
2.7. Maxion debug status monitor bus valids are sometimes incorrect	49
2.8. Maxion MISA.X value is incorrect	50
2.9. UltraSoC ME reports incorrect GPIO parameters in discovery response	51
2.10. SPIO UltraSoC BPAM module cannot read PLL/CRU registers	52
2.11. ETL2AXI bridge does not handle illegal requests from the SP	53
2.12. SP allows cache operations to be illegally performed on uncacheable space	54
2.13. sms_sft_ready/fail interrupts do not work before PLLs are configured	55
2.14. Perstn assert/deassert interrupts do not work before PLLs are configured	56
2.15. Scandump does not work due to incorrectly connected stop clock module	57
2.16. DFT does not have appropriate control of the eMMC PHY clock	58
2.17. Vault Scan Chain cannot be isolated	59
2.18. Prefetch CacheOp will hang Maxion	60
2.19. CSR tdata1 match field reset value is not 0, as defined in the RISC-V Debug Specification version 0.13	61
2.20. To be RISC-V compliant, tdata3 should be implemented as a register instead of trapping as an illegal instruction	62
2.21. TileLink to AXI signals BREADY and RREADY are not initialized at reset	63
2.22. Flush Op completion does not wait for ReleaseAck from TileLink	64
2.23. sfence generates a non-committed instruction fetch using mtvec	66
2.24. mtval CSR is written with an improperly sign-extended address	67
2.25. Fdiv.s and fsqrt.s with SNaN operands do not set Not Valid flag (fflags[NV])	68
2.26. Maxion PLL registers cannot be accessed when the Maxion Uncore Clock is running at target frequency	69
2.27. Maxion feature control bit to enable clearing of load reservation on voluntary eviction does not work	70
2.28. IOShire SPIO PLL0 Output Frequency Beats with the SPI ssi_clk	70

2.29. PTW accesses are not checked for access violations	72
2.30. Maxion does not prevent unprivileged access to its ESRs	73
2.31. Maxion PMPs allow barred accesses to leave core	74
3. Shire Cache	75
3.1. Neigh request hangs if shire cache full of I3_slave requests to same SCP address	75
3.2. L3_yield priority does not have sub-bank granularity	76
3.3. Performance Monitor Cycle Counter is counting incorrectly when there is no outstanding requests in Shire Cache Bank	77
3.4. SCP Fill requests from SCP source address using self shire id does not translate outgoing shire_id to actual shire_id	78
3.5. If 2 errors occur in the same cycle, one gets logged, but the multi bit does not get set.	78
3.6. Remote SCP Write Arounds and Cache Evicts are not ordered against other Remote SCP actions to the same address.	79
4. Memshire	82
4.1. Memshire East NoC voltage monitors are connected incorrectly	82
4.2. Diagnostic poisoning DDR data requires SW workaround	83
4.3. Changing DDR Controller frequencies requires SW workaround	84
4.4. Changing DDR Controller frequencies using a new PSTATE requires SW workaround	85
5. Pshire	86
5.1. PCIe Controller PCI_MSI_EXT_DATA_EN register write must be done by DW write accesses	86
5.2. PCIe Controller unable to enter loopback state in special use case	87
5.3. PCIe Controller changing data rate configuration from GEN4 to GEN3 or GEN2 has an issue	88
5.4. PCIe Power Management Functions are Unproven	89
6. General	90
6.1. HPD PLL hold time violations	90
6.2. DFT scan chain router hold time violations	92
6.3. HPDPLL and LVDPLL do not wait for LDO setup	93
7. ET-SOC A0 Erratum Template	94
7.1. <Template, enter here the title of the erratum>	94

1. Revision History

Version	Date	Author	Changes
v1.0	2019.12.10	Stephen Lee	Document created
...			
	2022.11.14	Jennie Weyant	Format and grammar review of sections 1.29 & 1.30
	2022.12.04	Daniel Yau	Added Section 5.4 regarding PCIe Power Management

2. Minion and Minion Shire

2.1. Denial of service in broadcast widget

ID: RTLMIN-2887 <https://esperantotech.atlassian.net/browse/RTLMIN-2887>

Owner: Xavier Revés

Block: Shire Channel, Uncached block

Description:

With the current architecture of the broadcast widget, there is a potential denial-of-service attack where a low-privileged Minion succeeds in preventing a higher-privileged Minion in the same Shire from using the broadcast widget.

Impact:

The impact is that broadcast operations can be blocked. However, the spec already states that SW has to coordinate to ensure that there are no problems with the 2-stage access to ESR to complete a broadcast operation. The issue may therefore appear only in the case of malicious or incorrectly implemented SW.

Workaround:

No workaround.

Fix Status:

Fix not done.

Fix for next silicon version.

2.2. TensorLoadSetupB does not support context switch

ID: RTLMIN-4535 <https://esperantotech.atlassian.net/browse/RTLMIN-4535>

Owner: Xavier Revés

Block: Minion DCache, TensorLoadSetupB FSM, Intpipe CSR

Description:

If a Minion must perform a context switch after a TensorLoadSetupB has started but before its pairing TensorFMA has finished, the state of the FSM cannot be read and stored to later restore it.

The SW in charge of context switching can cancel the TensorLoadSetupB to prevent the FSM from keeping its state, and the next process can read the pending data by doing a TensorFMA. However, after the TensorLoadSetupB operation has been cancelled, in general, it is not possible to re-start the operation before returning the context to the previous process.

Impact:

Context switching of processes is not possible using TensorLoadSetupB. If TensorLoadSetupB is not cancelled during process switching, the additional issue of data leakage may occur.

This could be considered a minor impact if it is assumed that processes will be switched only after they have finished their work or have been killed/cancelled.

Workaround:

There is no workaround apart from not doing context switching.

Fix Status:

Fix not done. We would need the following:

- New CSRs to expose the full state info needed for TensorLoadSetupB
- One bit in one of those CSRs to be able to "cancel" the current operation
- Ability to read the two CSRs by the M-code or S-code and write them back

This would allow true "context saving" and therefore a correct context switch operation across both threads using TensorLoadSetupB.

Fix for next silicon version.

2.3. Gather/Scatter instruction is missing some load/store operations since gsc_progress register is not updated

ID: RTLMIN-5374 <https://esperantotech.atlassian.net/browse/RTLMIN-5374>

Owner: Xavier Revés

Block: Minion DCache and Intpipe

Description:

This is a problem with the CSR register “gsc_progress” not being zero (i.e., it has been set because one of the sub-instructions of the Gather/Scatter [GSC] instruction got an exception) but being returned to zero as the GSC operation successfully ends.

The following GSC instruction, which takes the value in gsc_progress as the first sub-instruction to process at the ID stage, will not see the update since the last GSC sub-instruction won't update the value until the MEM stage, three cycles later.

The gsc_progress register is designed to speed-up the completion of GSC instructions that get an exception in any of its sub-instructions. The exception handler can resume a GSC instruction that gets an exception in one of its sub-instructions after fixing the cause of the exception or increasing the gsc_progress to skip the “bad” instruction. If the exception handler resumes the GSC instruction and a new GSC instruction comes after the current one that is executed within three cycles of completing the previous one, the issue will be hit.

Impact:

The exception handler cannot resume the GSC instructions that get an exception. From a SW perspective, this is a minimal impact since no exceptions are expected and if one occurs, it's likely that the thread will be terminated instead of continuing with the instruction.

Note that while using the gsc_progress feature or completing the GSC instruction that got an exception could be especially interesting when using pages for Virtual Memory (VM), this is not currently functional.

Workaround:

Do not resume GSC instructions that get an exception. Complete them step-by-step if needed or just kill the thread with the unexpected exception.

Fix Status:

Fix not done

2.4. Field CacheOp_Max in register ucache_control cannot be set to 0

ID: RTLMIN-5716 <https://esperantotech.atlassian.net/browse/RTLMIN-5716>

Owner: Xavier Revés

Block: Minion DCache, Cache operations FSM

Description:

This is a problem with the “unlimited” number of outstanding prefetch operations. When the number of outstanding operations reaches 32, the counter saturates. In the CSR register ucache_control, the “CacheOp_Max” field can be set to 0, which indicates that there is no limit in the number of outstanding requests.

However, if the number of requests that didn't receive a response exceeds 31, then some counters saturate (max count value is 31 when using the 5-bit counter) and subsequent counting is not correct. If the “unlimited” case has to be supported, then an “infinite” number of bits would be necessary to properly count the outstanding requests.

Impact:

The impact is minor. Even if this is a bug, it probably makes no sense to set this value to “unlimited”. The reason for configuring the maximum number of outstanding requests and the repetition rate is just to avoid flooding the L2 interface. For a practical implementation, it would therefore make sense to limit the maximum number of outstanding requests. Since the default value is 8, there is a wide margin up to 31 to cover a long latency in the responses.

Notice that software can still set the limit to values larger than the maximum number of lines that can be operated in a single configuration (16), and since it can use two different IDs and the TensorWait bits to check whether the previous operation with the same ID has ended or not, the rate of requests to L2 can still be kept to the level of saturating the interface when multiple Minions work simultaneously.

Workaround:

SW should set a value from 1 to 31 in the field CacheOp_Max of the CSR.

Fix Status:

Trivial fix. Implies re-defining the count limit for a '0' value.
Fix for next silicon version.

2.5. HART ESR address is not correctly checked

ID: RTLMIN-5985 <https://esperantotech.atlassian.net/browse/RTLMIN-5985>

Owner: Albert Navarro

Block: Minion core

Description:

The Minion APB interface only checks the 4 LSBs from the ESR address. This creates aliasing between the register addresses.

Impact:

Access to ESRs inside the hart ESR space that doesn't exist may result in accessing valid ESRs instead of returning an error.

This doesn't affect the functional mode since the only ESRs that we have in hart ESR spaces are only intended to be used by the Debugger and they are only accessible from the Debugger or the Service Processor.

Workaround:

Not needed.

Fix Status:

Postponed.

2.6. Enabling virtual memory can lead to undefined behavior

ID: RTLMIN-6001 <https://esperantotech.atlassian.net/browse/RTLMIN-6001>

Owner: Grigorios Magklis

Block: Minion core and Neighborhood

Description:

The Minion processor supports both the Sv39 and Sv48 virtual memory systems of RISC-V. However, there are a few known bugs in the current implementation that can create undefined behavior, including hanging the processor. Thus, it is advisable for SW to execute in “Bare” mode, where address translation is disabled.

Impact:

The impact is minor. Currently, SATP resets to zero so that VM is disabled (this configures “Bare” mode), and SW never writes it, so this erratum has no effect.

Workaround:

M-mode SW can set `mstatus.tvm=1` to make accessing SATP by S-mode trap to M-mode. This would allow M-mode to emulate SATP as hardwired to 0 for S-mode code, which effectively disables VM support. Also, M-mode SW should set `matp.mode=0` (default value at reset) and `matp.L=1` to disable M-mode virtual memory.

Fix Status:

Postponed.

2.7. When the "stride" value is set unrealistically high, address overflow may not be detected

ID: RTLMIN-6004 <https://esperantotech.atlassian.net/browse/RTLMIN-6004>

Owner: Xavier Revés

Block: Minion core

Description:

When the stride value for a tensor or cache operation (from register x31 according to the spec) is unrealistically high, the hardware implemented to detect the overflow may not work.

If both the base address and stride are in the same order of magnitude and they are combined (addition), the overflow logic may fail.

Overflow logic is simple and does the following:

```
addr_next = addr + stride;
```

```
overflow = addr[MSB] != addr_next[MSB];
```

The implementation assumes that the stride will never be extremely large. In the current implementation, if the stride does not have the MSB set to '1', the overflow detection always works. This means that the stride should be less than 0x8000_0000_0000, which is an already extremely high limit.

Impact:

Minor. Even if SW uses unrealistically high stride values, once the address falls within a memory region that cannot be accessed, the PMA will generate an exception. Overflow detection is not strictly necessary for Tensor or cache operations.

This issue cannot be hit if Virtual Memory is not activated.

Workaround:

Have SW use stride values lower than 0x8000_0000_0000.

Fix Status:

Postponed.

2.8. The mask value of contiguous vector stores that bypass L1 is ignored

ID: RTLMIN-2840 <https://esperantotech.atlassian.net/browse/RTLMIN-2840>

Owner: Grigorios Magklis

Block: Minion core and Neighborhood

Description:

Contiguous masked vector stores that bypass the L1 DCache may corrupt data in memory if the mask is not all 1s, because the implementation ignores the mask value and considers the mask to be all 1s. This impacts:

- FSWL.PS and FSWG.PS
- FSWL.PS, FSWG.PS, and FSW.PS when the L1 data cache is in bypass mode (via the neigh_chicken ESR).

Impact:

The impact is minor. Currently, our SW does not generate FSWL.PS nor FSWG.PS instructions, and we do not plan to disable the L1 DCache in normal operation.

Workaround:

In normal operation, SW should make sure that FSWL.PS and FSWG.PS are executed with M0 set to all 1s. When the L1 DCache is disabled, SW should make sure that FSWL.PS, FSWG.PS, and FSW.PS are executed with M0 set to all 1s.

Fix Status:

Postponed.

2.9. Using message port extensions can lead to undefined behavior

ID: RTLMIN-6089 <https://esperantotech.atlassian.net/browse/RTLMIN-6089>

Owner: Albert Navarro

Block: Minion core

Description:

The Minion processor supports a message system that allows for messages to be exchanged using a CSR and ESR interface. However, the feature is not verified and there are a few bugs that can lead to undefined behavior.

Impact:

The impact is minor. The SW still exchanges information using the memory or SCP, and the SW does not use message ports, so this erratum has no effect.

Workaround:

The SW can use the memory or the SCP to exchange information.

Fix Status:

Won't be fixed. This feature is not intended to be supported or be available in the future.

2.10. Tensor Conv clashes with a write to tensor_mask

ID: RTLMIN-6136 <https://esperantotech.atlassian.net/browse/RTLMIN-6136>

Owner: Xavier Revés

Block: Minion core

Description:

When the Tensor Conv FSM finishes computing a new value for the Tensor mask, it writes it into the tensor_mask CSR. If a CSR write to tensor_mask register happens in the same cycle, even if it is not considered for execution, it overwrites the tensor_mask content. The value computed by Tensor Conv is then lost.

This issue only appears when the write to tensor_mask happens exactly 16 cycles after Tensor Conv FSM is triggered by a write to tensor_conv_ctrl or tensor_conv_size CSRs.

Impact:

A subsequent Tensor operation may use the wrong tensor mask value, not taking the expected outcome from the Tensor Conv operation.

Workaround:

The SW has two possible workarounds:

1. After updating tensor_conv_* registers, ensure that there are at least 16 instructions before writing to the tensor_mask register.
2. If (1) cannot be guaranteed, do a read to the tensor_mask register before a write. If the Tensor Conv FSM is computing the tensor_mask register, the read instruction will be “replayed” and wait until Tensor Conv finishes without updating the tensor_mask. Once the Tensor Conv finishes, the read and then the write will safely happen.

Fix Status:

Won't be fixed. There is a simple and low-cost SW workaround.

2.11. Starting a TensorQuant operation may incorrectly update bit 4 (SCP not enabled) of the Tensor Error CSR for threads 0 and 1

ID: RTLMIN-6215 <https://esperantotech.atlassian.net/browse/RTLMIN-6215>

Owner: Sebastià Tortella

Block: Minion core

Description:

When a TensorQuant operation requiring the L1 Scratchpad (SCP) is started, but the SCP is not enabled, Tensor Error bit 4 should be set to indicate that the operation cannot be carried out.

The Tensor Error register has one version for thread 0 and another for thread 1. In some cases, when starting a TensorQuant operation, the previously mentioned error bit may be set for thread 1 instead of thread 0.

Impact:

Software for thread 1 should not be impacted since it cannot start Tensor Operations and thus should never look at this bit.

Software for thread 0 may not see the error bit set, which would give it information about the cause of the TensorQuant operation not being performed. However, before performing any TensorQuant operation, SW must ensure that the SCP is set because at least a TensorLoad operation is necessary to write data into the SCP. If the SCP is not set, not even the necessary Tensor operations previous to TensorQuant will be performed.

A TensorLoad operation to write data into the SCP will not properly set the Tensor Error bit if SCP is not set.

Workaround:

The SW needs to enable SCP if it wants to perform any Tensor operation requiring the SCP.

Fix Status:

Won't be fixed. The SCP must always be enabled when SW performs Tensor operations.

2.12. Branch Taken events for performance counters are not correctly generated

ID: RTLMIN-6221 <https://esperantotech.atlassian.net/browse/RTLMIN-6221>

Owner: Sebastià Tortella

Block: Minion core

Description:

The generation of the “Branch Taken” event is not correct, as it is not properly validated and may stay active for multiple cycles each time that a branch can be taken.

Impact:

Selecting the “Branch Taken” event for the Performance Monitor Unit counters will not produce correct counting values.

Workaround:

None.

Fix Status:

Won't be fixed. It is not critical that these events be counted.

2.13. Minion data breakpoints not working properly on CacheOps and TensorLoads/Stores

ID: RTLMIN-6253 <https://esperantotech.atlassian.net/browse/RTLMIN-6253>

Owner: Sebastià Tortella

Block: Minion core

Description:

Data breakpoints triggered by asynchronous DCache operations (CacheOps and TensorLoads and Stores) don't work as expected and might halt the incorrect thread.

Impact:

Configuring a data breakpoint on addresses that are used by a CacheOp or a TensorLoad/Store can halt the incorrect thread of the Minion, that is, it can halt the thread that didn't perform the CacheOp or TensorLoad/Store instead of the intended one.

Workaround:

None.

Fix Status:

Postponed.

2.14. VPU not connected to Minion Shire warm reset

ID: RTLMIN-6282 <https://esperantotech.atlassian.net/browse/RTLMIN-6282>

Owner: Jordi Muñoz

Block: Minion core

Description:

The Minion VPU is not connected to the Minion Shire warm reset, only to the global cold reset.

At some point, the warm reset feature was deprecated and hence, deprioritized from the DV test plan.

Impact:

If a warm reset is applied while the VPU is active, this might cause some internal VPU parts to not be properly cleared or the Tensor FSMs to hang waiting for data from the core.

Given that this feature has not been fully verified, there might be other flaws in the system, so we cannot trust that it works.

Workaround:

Use the cold reset.

The SW workaround will be evaluated after the DV team confirms that the feature works.

Fix Status:

Postponed

2.15. Part of a misaligned access may execute with the wrong privilege

ID: RTLMIN-6398 <https://esperantotech.atlassian.net/browse/RTLMIN-6398>

Owner: Grigorios Magklis

Block: Minion core

Description:

When a misaligned memory access (load or store) spans two cache lines, it is executed as two micro-operations in the Minion core. The expected behavior is that these two micro-operations execute with the effective privilege level that was active at the time the instruction was issued. However, in A0, if a younger ECALL, SRET, or MRET instruction changes the hart's privilege mode before the 2nd micro-operation executes, the latter will execute with the new privilege mode instead of the expected privilege mode.

Impact:

Misaligned accesses are only permitted for addresses corresponding to the L2 scratchpad and DRAM. The L2 scratchpad is accessible by all privilege levels, therefore L2 scratchpad accesses are not impacted by this issue. The DRAM address space is divided into regions where accesses to some regions are permitted only if originated from M-mode, and to some other regions only if originated from either M-mode or S-mode. However, by design, there is no security issue due to privilege escalation when accessing the DRAM because if an agent has sufficient privileges to access address X, it has sufficient privileges to access address Y when $Y > X$. This means that if the 1st micro-operation has sufficient privileges, so does the 2nd; therefore, the 2nd micro-operation cannot generate a page fault exception unless the 1st one would as well. However, when virtual address translation is disabled and both micro-operations access the M-mode or S-mode region of the DRAM, and an xRET instruction downgrades the privilege of the 2nd access, then the 2nd access will incorrectly generate an access fault exception (which will be converted to a Bus error interrupt).

Workaround:

Privilege software should either avoid misaligned accesses that span cache lines or make sure that any possibly affected micro-operations have been issued before executing the xRET instruction. The latter can be achieved by a two-instruction sequence: a FENCE instruction and a naturally aligned memory instruction (whose result need not be used) that forces the FENCE to stall execution before the xRET instruction. For example, "fence; lb x0, (sp); sret".

Fix Status:

Not fixed.

2.16. Exceptions on misaligned accesses may not release Minion resources

ID: RTLMIN-6397 <https://esperantotech.atlassian.net/browse/RTLMIN-6397>

Owner: Grigorios Magklis

Block: Minion core

Description:

When a misaligned memory access spans two cache lines, it is executed as two micro-operations. If virtual address translation is disabled and the 2nd micro-operation generates an access fault exception, then the expected behavior is to raise a Bus error interrupt and free all resources allocated. In A0, the replay queue entry allocated for the micro-operation is never released and can no longer be allocated by future instructions. The immediate effect of this is reduced performance, but if this happens multiple times and all replay queue entries become un-allocatable, the Minion will deadlock. Also, if an entry becomes un-releasable and un-allocatable and a FENCE instruction is executed, the FENCE instruction will stall the issuing hart forever. The only method to recover from this situation is to reset the Minion core.

Impact:

The 2nd micro-operation can generate an access-fault exception in the following situations:

- Misaligned access that is partially inside and partially outside the L2 scratchpad region (at the highest address boundary for the L2 block accessed).
- Misaligned access that is partially inside and partially outside the DRAM region (at the highest DRAM address boundary).
- Misaligned access that is partially inside and partially outside the “OS region” in the DRAM (at the lowest OS region address boundary) and mprot[3]==1.
- The cases described in [RTLMIN-6398](#) above.

All of the above cases, except the ones described in [RTLMIN-6398](#), are considered software errors.

Workaround:

For the above cases that correspond to software errors, software should take precautions because they are susceptible to this hardware bug. [RTLMIN-6398](#) describes the SW workaround for the non-error cases.

Fix Status:

Not fixed.

2.17. The SBM may drop ESR write responses if there is a broadcast inflight and BPAM accesses to the same endpoint

ID: RTLMIN-6408 <https://esperantotech.atlassian.net/browse/RTLMIN-6408>

Owner: Albert Navarro

Block: Minion Shire

Description:

For the SBM broadcast, which writes to the same ESR in all the Neigh/SCs, the SBM assumes that the WR requests can't be executed while the broadcast is inflight. Unfortunately, this is not correctly enforced in A0. If two or more write requests can execute while the broadcast is inflight, their responses will try to go through the AXI_B arbiter and pop the next cycle since the SBM thinks they are responses from the broadcast. If the two responses come in the same cycle, one of them will be lost.

So, the write requests can execute while the broadcast is inflight. However, the broadcast responses need to come in different cycles, which only happens when the BPAM is accessing the same endpoint.

Impact:

It's not safe to use the BPAM to read/write the Neigh/SC ESR banks while there are broadcast requests and write requests inflight. BPAM requests, write requests, and broadcasts need to target the same endpoint. In A0, that only happens during SoC initialization.

Workaround:

Avoid concurrent use of the BPAM + write requests + broadcasts to the same endpoint.

Fix Status:

Not fixed.
Will be fixed in the Next Gen.

2.18. Resume state machine only works for Minion 0

ID: RTLMIN-6455 <https://esperantotech.atlassian.net/browse/RTLMIN-6455>

Owner: Joel Sanchez

Block: Minion Shire

Description:

An external debugger can apply different run control operations (halt, resume, or reset) on the Minion cores. The resume request that arrives at a given hart is deasserted when the hart's state changes to running, or, if the hart is already running, it is deasserted immediately.

In A0, the state machine that deasserts the resume request of a hart H erroneously looks at the state of the corresponding hart of Minion 0 of H 's neighborhood instead of looking at H 's state (i.e., if hart H is even, it looks at hart 0 of the neighborhood, and if hart H is odd, it looks at hart 1 of the neighborhood). This means that, independent of H 's state, when hart H receives a resume request, it will not be deasserted until hart 0 or 1 (for even or odd H , respectively) of H 's neighborhood is running. While the resume request of a hart is active, the hart cannot be halted.

If an external debugger attempts to resume hart H , while hart 0 or 1 (for even or odd H , respectively) of H 's neighborhood is not running, then H will resume execution, but its resume request will not be deasserted and H will not be able to be halted again until hart 0 or 1 (for even or odd H , respectively) of H 's neighborhood is running. A hart in this state will ignore all halt attempts made by hardware triggers, EBREAK instructions, halt-on-reset, or hardware-single-step.

Impact:

It's not safe to perform a resume run control operation on hart H while hart 0 or 1 (for even or odd H , respectively) of H 's neighborhood is not running. Hart 0 or 1, from H 's neighborhood, must be running to ensure resume run control operation is performed as expected.

Workaround:

The external debugger should never send a resume request to a hart H that is not hart 0 or 1 of a neighborhood without also sending the same resume request to hart 0 or 1 (for even or odd H , respectively) of the neighborhood or when hart 0 or 1 (for even or odd H , respectively) of the neighborhood is unavailable.

Sometimes the external debugger may need to resume a particular hart H that is not hart 0 or 1 of a neighborhood while the corresponding hart 0 or 1 of the neighborhood is halted. For

example, when resuming execution after a software breakpoint is hit, the debugger usually performs the following actions:

1. Remove the SW breakpoint.
2. Single step hart H (the hart that hit the SW breakpoint).
3. Restore the SW breakpoint.
4. Resume all harts.

In this case, the debugger can perform the following actions instead:

1. Remove the SW breakpoint.
2. Remove any HW breakpoints on the corresponding hart 0 or 1 of the neighborhood.
3. Set a HW breakpoint on the corresponding hart 0 or 1 of the neighborhood with the DPC value as the breakpoint address. This will halt the hart immediately after resuming without executing any instructions.
4. Configure hart H to hardware-single-step.
5. Resume both hart H and the corresponding hart 0 or 1 of the neighborhood. The latter will halt immediately, without executing any instructions, while the former will step over the SW breakpoint and the halt.
6. Restore any HW breakpoints on the corresponding hart 0 or 1 of the neighborhood.
7. Restore the SW breakpoint.
8. Resume all harts.

Fix Status:

Not fixed.

2.19. Hart can execute unexpected instructions in debug mode

ID: RTLMIN-6452 <https://esperantotech.atlassian.net/browse/RTLMIN-6452>

Owner: Albert Navarro

Block: Minion Shire

Description:

The debugger makes use of the program buffer to load multiple instructions on the core in such a way that they are executed to read and manipulate the state of the hart while the core is halted. The program buffer is accessed through requests to internal Minion registers (ABSCMD, NXPROGBUF0, NXPROGBUF1, AXPROGBUF0, and AXPROGBUF1) which are mapped as ESRs.

The program buffer has four entries of 32 bits. Up to four 32-bit instructions or eight 16-bit instructions are supported.

If the debugger tries to use the NXPROGBUF* registers to access entries 2 and 3 of the program buffer, the core will execute undefined instructions from the FE buffer without notifying the debugger (busy bit).

Impact:

It's not safe to use NXPROGBUF0 or NXPROGBUF1 ESRs since the core will execute an undefined instruction sequence after the debugger accesses these ESRs.

Workaround:

Make use only of ABSCMD such that the debugger loads two instructions and then executes instead of loading multiple instructions in the program buffer and then executing.

In case it is not enough to use ABSCMD and the debugger must use the upper part of the program buffer, it can write the upper part using the following sequence:

1. Load the ABSCMD with ebreaks before loading the rest of the instructions to ensure that they are not executed through performing the following sequence of instructions:
 - Write ABSCMD with ebreaks
 - ebreaks return the control-flow back to the debugger, so nothing will be executed
 - Write AXPROG0 with the third instruction.
 - Write AXPROG1 with the fourth instruction.
 - Write ABSCMD with the instruction 0 and 1 of the sequence.

Fix status:

Not fixed

2.20. Bad mtval after debug exception for PC that has fetch AF in the second half of the instruction

ID: RTLMIN-6448 <https://esperantotech.atlassian.net/browse/RTLMIN-6448>

Owner: Sebastià Tortella

Block: Minion Shire

Description:

When the debug CSRs are configured to have a breakpoint exception (cause=3) for a program counter that is not aligned to a 4B boundary that also has a fetch access fault because of the 2nd half of the instruction, the mtval CSR has an incorrect value: it should be PC, but it is PC+2.

Impact:

Since we are not using virtual memory, the access fault has to come from the PMA. If so, DRAM_END - 1 is the only misaligned address that can trigger this issue. Because it would be unusual to set a breakpoint at this address and there is an easy work around, the impact is very limited.

Workaround:

If the software programs a fetch breakpoint exception, instead of reading the program counter from mtval, it can read it from mepc.

Fix status:

Not fixed.

2.21. Amocmpswap x31 implicit dependency not correctly tracked

ID: RTLMIN-6469 <https://esperantotech.atlassian.net/browse/RTLMIN-6469>

Owner: Sebastià Tortella

Block: Minion Shire

Description:

The amocmpswap instructions have an implicit dependency with x31 that is not properly tracked when x31 is written by some long latency instructions.

Specifically, if x31 is written by a load instruction that misses in L1, there are up to three cycles where an incorrect amocmpswap could be issued that would use the previous value of x31 instead of the one resulting from the load. Similarly, if x31 is the result of a multiply or divide instruction, there is one cycle where an incorrect amocmpswap could be issued.

Impact:

The instruction is rarely used and the compiler won't generate it by itself: it has to be written in assembly (e.g. embedded in an extended asm block inside C/C++ code). So it is feasible to use the following workaround.

Workaround:

Adding an arithmetic instruction between the instruction writing x31 and the amocmpswap. This arithmetic instruction has to use x31 without modifying its value. For instance,

```
ld x31, offset(rs)
amocmpswap rd, rs2, (rs1)
```

becomes:

```
ld x31, offset(rs)
addi x31, x31, 0
amocmpswap rd, rs2, (rs1)
```

Fix status:

Not fixed

2.22. Writes to PMU registers may happen twice

ID: RTLMIN-6488 <https://esperantotech.atlassian.net/browse/RTLMIN-6488>

Owner: Xavier Revés

Block: Minion Intpipe CSR file

Description:

Any instruction that writes to CSRs will write the same value once or twice when it's being used to update the PMU counters. It will write it once if the previously accessed PMU register was the same as the one being accessed by the current instruction. Otherwise, it will write it twice.

This is not a problem when using CSR write-only instructions, as the written value will be the same and the behavior will be equivalent to two consecutive write instructions. For instructions that do atomic reads and writes, however, the first (unwanted) write will update the PMU register and the read operation will not return the former PMU register value but rather the value set by the first write.

Impact

Trivial. Very simple workaround exists.

Workaround:

Do not use atomic read and write CSR instructions; only use single read or write CSR instructions.

<code>csrr rd, csr</code>	<code>csrrs rd, csr, x0</code>	Read CSR
<code>csrw csr, rs</code>	<code>csrtw x0, csr, rs</code>	Write CSR
<code>csrs csr, rs</code>	<code>csrrs x0, csr, rs</code>	Set bits in CSR
<code>csrc csr, rs</code>	<code>csrrc x0, csr, rs</code>	Clear bits in CSR
<code>csrwi csr, imm</code>	<code>csrrwi x0, csr, imm</code>	Write CSR, immediate
<code>csrsi csr, imm</code>	<code>csrrsi x0, csr, imm</code>	Set bits in CSR, immediate
<code>csrci csr, imm</code>	<code>csrrci x0, csr, imm</code>	Clear bits in CSR, immediate

In the case that two consecutive writes are an issue even for single write CSR instructions, read the PMU register first and then write it. This sequence ensures that the write will happen only once.

The access control to the PMU registers should be separated per thread. Then, two threads accessing the PMU registers simultaneously will not notice any cross effect.

Fix Status:

Not fixed.

2.23. Simultaneous reading from both Minion threads of the PMU registers may return incorrect data to the second thread doing the read

ID: RTLMIN-6496 <https://esperantotech.atlassian.net/browse/RTLMIN-6496>

Owner: Xavier Revés

Block: Minion Intpipe CSR file

Description:

Each time that a thread changes the PMU register to read, there is an update to the read pointer for that thread. The update to the pointer of one thread will collide with the update of the pointer of the other thread if the second reading happens within a window of three cycles after the first.

When a thread in a Minion reads a PMU register, the returned value may be incorrect. This may happen if the other thread in the same Minion is reading different PMU registers at the same time. However, if any of the threads always reads the same register, the read value is always correct.

Impact

Minor. A workaround exists and using the PMU counters is not critical.

Workaround:

If both Minion threads have to “randomly” access the PMU registers:

- Use a “software mutex” or enter into “exclusive mode” before reading a PMU register
- Make 4 “atomic” consecutive reads to the register (i.e., without any interruption of more than ~11 cycles between the first and the last read).
 - Interrupts must be disabled
 - Instructions must all be within a half cache line

If only one thread in a Minion does random access to the PMU registers while the other always reads the same register (or no register):

1. No workaround is needed. However, ensure that the first read of the “always read” register is safely done (e.g., with a “software mutex”, entering into “exclusive mode” before reading a PMU register, passing a credit, etc.).

Fix Status:

Not fixed.

2.24. Debug access to the L1 ICache tag memory incorrectly enables simultaneous access to the L0 microcache 0 memories

ID: RTLMIN-6509 <https://esperantotech.atlassian.net/browse/RTLMIN-6509>

Owner: Ildefonso Gomariz

Block: Neighborhood's L1 ICache on Minion Shire and SP's L1 ICache on IOShire

Description:

The debug access through the APB to the L1 ICache tag memory doesn't work properly. The APB access to the L0 microcaches does not check bit 12 of the address, which selects between the two accesses, thus an access to the L1 ICache tag memory will also incorrectly enable a simultaneous access to the L0 microcaches.

More specifically, an access to the L1 ICache tag memory will also generate an access to the L0 microcache 0, but not vice versa. Depending upon the accessed set, a different L0 microcache 0 memory will be accessed in parallel (either data, tags, or error flags).

As a result, on a read access to the L1 ICache tag memory, the read data will be an OR'ed version of the data returned by both memories. On a write access, the L0 microcache 0 memories will be incorrectly overwritten as well.

Impact:

Trivial. It only affects debug accesses and there is an easy SW workaround.

Workaround:

Read access to L1 ICache tags:

1. Read contents from the equivalent address of Ucache 0 (i.e., `addr_ucache0 = addr_l1tag | 0x1000`) and save it
2. Overwrite the equivalent address of Ucache 0 with 0s
3. Read contents from the L1 ICache tag memory. Read data should be correct, as it is being OR'ed with 0s
4. Write the equivalent address of Ucache 0 with the original contents that were saved previously

Write access to L1 ICache tags:

1. Read contents from the equivalent address of Ucache 0 (i.e., `addr_ucache0 = addr_l1tag | 0x1000`) and save it
2. Write contents to the L1 tag memory. Ucache 0 will also be overwritten
3. Write the equivalent address of Ucache 0 with the original contents that were saved previously

Fix Status:

Not fixed.

2.25. Write to the Tensor mask does not wait for previous tensor_conv_size

ID: RTLMIN-6520 <https://esperantotech.atlassian.net/browse/RTLMIN-6520>

Owner: Xavier Reves

Block: Minion Intpipe CSR file

Description:

The modification of the tensor_mask register can happen in two different ways: 1) through changes to the tensor_conv registers and 2) writing directly to the tensor_mask register.

When an atomic CSR read/write instruction is issued (let's say this is instruction I1) to read and update the tensor_mask register, the register might be "busy" because of an ongoing tensor_conv operation (let's say this is instruction I0). In this case, the I1 instruction is going to wait until the instruction I0 ends.

However, the "write" part of the I1 instruction will be executed and the tensor_mask register will be updated. If this happens just after the I0 instruction has written its output to the tensor_mask register, when the I1 instruction resumes the execution, the "read" value will be that which was written by the same I1 instruction.

This is a side-effect of RTLMIN-6136 "Tensor Conv clashes with a write to tensor_mask" when using atomic CSR read/write operations.

Impact:

Trivial. A very simple workaround exists.

Workaround:

Do not use atomic CSR read/write instructions on the tensor_mask register. Use a read instruction first and then the write instruction.

Fix Status:

Not fixed.

2.26. For CacheOps, the Tensor mask for one thread can leak and be used as the Tensor mask for the other thread

ID: RTLMIN-6537 <https://esperantotech.atlassian.net/browse/RTLMIN-6537>

Owner: Xavier Reves

Block: Minion Intpipe CSR file

Description:

A thread may use the Tensor Conv FSM to compute the Tensor mask for any Tensor or cache operation. There are two CacheOp FSMs, one that handles the L1 operations and the other, the L2 operations. Both FSMs can operate simultaneously, started by either the same or different threads.

When one of the CacheOp FSMs is started by any thread (let's say thread A), if the Tensor mask is going to be used before the Tensor Conv FSM has finished the computation of the mask, the CacheOp FSM will wait until the Tensor mask for the initiating thread (A) is updated by the Tensor Conv FSM.

During this waiting period, if the other thread (let's say thread B) starts the other CacheOp FSM, the pointer in the CSR file that selects the Tensor mask to send to the CacheOp FSMs will be switched to point to this thread (B) Tensor mask register. A few cycles later, when the Tensor Conv completes the computation of the Tensor mask for thread A and the CacheOp FSM that was started by this same thread resumes, the value of the taken Tensor mask will be the one associated with thread B.

Impact:

Trivial. A very simple workaround exists.

Workaround:

If a thread uses the Tensor Conv FSM to compute the Tensor mask for a CacheOp, it should read the Tensor mask register before starting the CacheOp. The workaround is identical to the one for RTLMIN-6136. The read instruction will be "replayed" and wait until Tensor Conv FSM finishes without updating the Tensor mask. Once the Tensor Conv ends, the read will safely happen and the CacheOp will use the Tensor mask associated with the thread that started the operation.

Fix Status:

Not fixed.

2.27. AndOrTreeL2 ‘allhalted’ and ‘allrunning’ fields are incorrect when there are no harts selected

ID: RTLMIN-5899 <https://esperantotech.atlassian.net/browse/RTLMIN-5899>

Owner: Albert Navarro

Block: IOShire miscellaneous ESRs

Description:

The AndOrTreeL2 allhalted and allrunning fields don't work properly when there are no harts selected in the system. The spec defines these two fields as follows:

- **allhalted:** Set if AndOrTreeL1.allhalted is set for each Minion Shire in the system with AndOrTreeL1.anyselected asserted.
- **allrunning:** Set if AndOrTreeL1.allrunning is set for each Minion Shire in the system with AndOrTreeL1.anyselected asserted.

When there is no hart selected in the system, these two fields should be '0', but instead they are '1' regardless of the state (halted/running) of the harts.

Impact:

The debugger will see harts halted and running simultaneously. In the case that the debugger doesn't check if there are harts selected, it may perform incorrect actions based on the reported state.

Workaround:

When the debugger reads the AndOrTreeL2 ESR and wants to check one or both of these fields, it must first check that they are both '1' and convert them to '0' in case it is needed. The debugger should follow the following expression:

```
allrunning = AndOrTreeL2.allrunning && !(AndOrTreeL2.allrunning && AndOrTreeL2.allhalted);  
allhalted  = AndOrTreeL2.allhalted  && !(AndOrTreeL2.allrunning && AndOrTreeL2.allhalted);
```

Fix Status:

Not fixed.

2.28. Intensive usage of UC operations in one Minion thread may produce starvation to the other thread

ID: RTLMIN-6614 <https://esperantotech.atlassian.net/browse/RTLMIN-6614>

Owner: Xavier Revés

Block: Minion

Description:

In the DCache, the Uncached (UC) memory accesses are blocking future memory accesses from any thread. When a thread is continuously doing UC accesses, the DCache Replay Queue (RQ) will likely become full of these pending accesses. Once one of these UC instructions completes, the next memory instruction that enters the DCache can be for thread 0 or thread 1. If the thread that gets access to the DCache is the same that floods the RQ, then the other thread has to keep waiting. When this situation repeats over and over, one thread will suffer starvation.

This is only going to happen if the number of cycles it takes for all the UC requests to complete has identical parity (odd/even) and the DCache accepts a new memory instruction that has the same thread as the one that is scheduled at the ID stage. If there is some jitter in this number of cycles, then sooner or later the other thread will gain access to the DCache.

In real scenarios with variability in the UC accesses, this situation is unlikely to happen.

Impact:

One of the threads cannot progress while the other thread is doing continuous UC accesses.

Workaround:

Given that the blocked thread has no option to modify the instruction pattern of the other thread, there is no workaround apart from avoiding infinite loops with UC accesses in any thread.

Fix Status:

Not fixed.

2.29. Minion VPURF Timing Path

ID: FV-259-6614 <https://esperantotech.atlassian.net/browse/FV-259>

Owner: Darren Jones

Block: Minion

Description:

When a write happens to the VPU register file (VPURF), this write requires two cycles instead of one.

Impact:

Because of logic outside of the VPURF, when a read happens to a VPURF register in the same cycle as a write to the same register, the data will be bypassed and the proper value (the written value) will be read. However, if that register is read in the cycle after it is written, incorrect data may be read.

Workaround:

Due to different pipeline behaviors, workarounds for different types of instructions that write to the VPURF are provided in the following table:

Type	Description	Instruction
A	Load instructions (vector or scalar FP) Transcendental Ops	FP Memory Loads: flq2, flw* FP Atomic Memory Ops: famo*. * FP Gather Ops: fg* FP Broadcast from Memory: fbc.ps FP Transcendentals: fexp.ps, flog.ps, frcp.ps
B	Integer computation instructions, move instructions	Cubemap ops: cube* Packed Int Computational Ops: fadd.pi, faddi.pi, fand.pi, fandl.pi, fnot.pi, for.pi, fsll.pi, fslli.pi, fsra.pi, fsrai.pi, fsrl.pi, fsrli.pi, fsub.pi, fxor.pi Packed Int Compare Ops: feq.pi, fle.pi, flt.pi, fltu.pi, fmax.pi, fmaxu.pi, fmin.pi, fminu.pi FP Broadcast from Register/Immediate: fbc.pi, fbc.ps, fbcx.ps FP Move Ops: fcmov.ps, fcmovm.ps, fmv.s.x/fmv.w.x, fpackreph.pi, fpackreph.pi, fswizz.ps Simple Conversions: fsat8.pi, fsatu8.pi, fsgnj.* , fsgnjn.* , fsgnjx.*
C	FP computation instructions (vector or	FP Compute: fadd.(ps,s), fclass.ps, ffrc.ps, fmadd.(ps,s), fmsub.(ps,s), fmul.(ps,s),

Type	Description	Instruction
	scalar FP)	fnmadd.(ps,s), fnmsub.(ps,s), fround.ps, fsub.(ps,s) Packed Int Multiply: fmul.pi, fmulh.pi, fmulhu.pi FP Conversions: fcvt.*.* FP Comparisons: feq.ps, fle.ps, flt.ps, fmax.(ps,s), fmin.(ps,s)
D	Tensor FMA instructions (internal operation)	TensorFMA32, TensorFMA16A32
E	TensorQuant (internal operation)	TensorQuant
F	Tensor Op ➡ NonTensorOp	TensorFMA32, TensorFMA16A32, TensorIMA8A32, TensorQuant, TensorRecv, TensorReduce, TensorBroadcast
G	TensorRecv ➡ TensorOp	TensorRecv/TensorReduce/TensorBroadcast ➡ TensorFMA32/TensorFMA16A32/TensorIMA8A32/TensorQuant

NOTE: VPU instructions can have up to three input operands read from the VPURF, each one of which may have been produced by a Type A, B, or C instruction. Therefore, each operand is subject to each of the Type A, B, and C workarounds.

2.29.1. Workaround for Type A

In the case of (A), the load may hit or miss in the cache subsystem, so the exact timing of the VPURF write relative to subsequent instructions is not known. Transcendental operations use iterative microops requiring up to 29 cycles. Therefore, to avoid this problem, you must first force the read dependency to resolve, and then insert an instruction after the dependent instruction as follows:

1	flq2 f0, (mem)	# Any instruction listed for Type A above
2	<non-f0-inst>	# 0 or more instructions, none of which use f0 as an operand
3	fmv.x.w x0, f0	# This instruction is inserted to force the dependency on f0 to resolve
4	<non-f0-inst>	# This can be any instruction that does not use f0 as an operand
5	<inst>	# This instruction (and any subsequent) will read f0 correctly from the VPURF

I3 is inserted as part of the above workaround. It is recommended that I3 be inserted two instructions before I5. In this case, I4 is actually the original instruction that was just before

I5. Thus, only I3 would have been inserted for the workaround. If, in the original code, I5 immediately follows I1, then you must also insert a NOP as I4.

NOTE: It is strongly recommended that the inserted I3 be exactly `fmv.x.w` with a destination register of `x0`. This enables checkers and assertions to detect this inserted instruction as part of the workaround. Since `x0` is used, this is effectively a NOP and would not be used in normal functional code.

2.29.2. Workaround for Type B

In the case of (B), SW can avoid the problem if (1) there are at least eight non-dependent instructions between the Type B instruction that writes to the VPURF and the one that reads that same register from the VPURF, or (2) there is a taken branch or call or ret instruction. If neither of these conditions can be met, then SW can use the following workaround:

1	<code>fadd.pi f0, f1, f2</code>	# Any instruction listed for Type B above
2	<code>flem.ps m1, f1, f1</code>	# This instruction is inserted. Any unused mask register can be utilized
3	<code><non-f0-inst></code>	# 0 to 7 instructions, none of which use f0 as an operand
4	<code>mov.x.m x0</code>	# This instruction is inserted and resolves the dependency
5	<code><inst></code>	# This instruction (and any subsequent) will read f0 correctly from the VPURF

In the above workaround, I2 and I4 are inserted; they force the pipeline to complete I1 and writeback results to the VPURF. To maximize performance, I2 should be the first instruction after I1 and I4 should be inserted just before I5. You can insert up to 7 non-dependent instructions between these two. In I2, any mask register can be used, but it should be a mask that is not currently being used, since the contents will be overwritten.

The above workaround requires an unused mask register. If a mask register cannot be used, the following workaround (which requires an unused general purpose register) could also be used:

1	<code>fadd.pi f0, f1, f2</code>	# Any instruction listed for Type B above
2	<code>fclass.ps f3, f3</code>	# This instruction is inserted. Any unused FP register can be utilized
3	<code><non-f0-inst></code>	# 0 to 7 instructions, none of which use f0 or f3 as an operand
4	<code>fmv.x.w x0, f3</code>	# This instruction is inserted and resolves the dependency

5	<inst>	# This instruction (and any subsequent) will read f0 correctly from the VPURF
---	--------	---

The above workaround is very similar to the first workaround, except that I2 and I4 are different instructions using an FP reg to force the dependency instead of a mask register. All other guidelines from the above description apply.

2.29.3. Workaround for Type C

In the case of (C), SW can avoid the problem if (1) there are at least eight non-dependent instructions between the Type C instruction that writes to the VPURF and the one that reads that same register from the VPURF, or (2) there is a taken branch or call or ret instruction. If neither of these conditions can be met, then SW can use the following workaround:

1	fmadd.ps f0, f1, f2, f3	# Any instruction listed for Type C above
2	<non-f0-inst>	# 0 to 6 instructions, none of which use f0 as an operand
3	fmv.x.w x0, f0	# This instruction is inserted to force the dependency on f0 to resolve
4	<non-f0-inst>	# This can be any instruction that does not use f0 as an operand
5	<inst>	# This instruction (and any subsequent) will read f0 correctly from the VPURF

In the above workaround, I3 forces the pipeline to complete I1. I4 must be present to allow the writeback to the VPURF to complete.

2.29.4. Workaround for Type D

In the case of (D), the bug is triggered by the internal microsequencing of the operation itself. This happens only for certain Matrix A sizes. Therefore, SW can avoid the problem if they avoid the following type of instructions:

- Instruction= TensorFMA32 or TensorFMA16A32, AND
- ACOLS > 0: xs[50:47] != 4'b0000, AND
- If matrix B is coming from the L1 Scratchpad (TENB==0)
 - AROWS== 3: xs[54:51] == 4'b0011
- If matrix B is coming from Memory (TENB==1)
 - AROWS <= 3: xs[54:53] == 2'b00

For instance, if software could pad the A matrix such that it could set the Matrix A rows==4 instead of 3, that would avoid the problem.

2.29.5. Workaround for Type E

TensorQuant is another instruction that can trigger the bug due to internal microsequencing. Each of the TensorQuant transformations work correctly when run individually. However, TensorQuant can run up to 10 transformations serially. The bug can be triggered between these transformations. The following table shows the configurations of the A matrix size that can cause the issue (NOTE: the oldest/earliest transformation is TXF0 and the youngest/latest transformation is TXF9):

AROWS (xs[54:51])	ACOLS (xs[56:55])	Notes
0	0,1	
1	2,3	
2	0,1	
	2,3	Only applies when the older transformation is FP32_MUL* or INT32_ADD*
3	all	
5	0,1	Only applies when the older transformation is FP32_MUL* or INT32_ADD*
7	0,1	
	2,3	Only applies when the older transformation is PACK128

To workaround this issue, SW must split the offending transformations into two separate TensorQuant instructions instead of one combined TensorQuant.

2.29.6. Workaround for Type F

When a Tensor Op writes to the VPURF and the next VPU operation is a non-tensor operation (Scalar FP, Packed Single [PS], or Packed Integer [PI]) instruction, the bug can be triggered if the final register written by the TensorOp matches one of the input operands for that next VPU instruction. In this case, SW can use the following workaround:

1	<TensorOp>	# Any instruction listed for Type F above
2	<non-vpu-inst>	# 0 or more instructions, none of which are Tensor Ops, scalar FP, PS, or PI instructions
3	fmv.x.w x0, f0	# This instruction is inserted to force the VPU pipeline to writeback its final result

4	<vpu-inst>	# This can be any non-Tensor instruction that reads the VPURF. This instruction (and any subsequent) will read fp registers correctly from the VPURF
---	------------	--

Each Tensor Op uses a microsequencer to perform microinstructions. The final register written by each Tensor Op is given by the formula in the following table. Alternatively, SW can simply insert the above `fmv.x.w` instruction without regard to the register being written.

Tensor Operation	Final Register Written	Notes
TensorFMA32 TensorFMA16A32 TensorIMA8A32	if(BCOLS<=1) { AROWS*2 } else { AROWS*2 + 1 }	Only needed when <code>tensor_mask</code> is not used (<code>MSK==0</code>) or when the <code>tensor_mask</code> for the final row is not masked (<code>tensor_mask[AROWS]==0</code>) For TensorIMA8A32, the bug only happens when the result is written into the VPURF (<code>DST==1</code>)
TensorQuant	if(ACOLS<=1) { (FREG+AROWS*2)%32 } else { (FREG+AROWS*2+1)%32 }	
TensorRecv TensorReduce TensorBroadcast	(FREG+COUNT-1)%32	Only needed when <code>COUNT>0</code> For TensorReduce and TensorBroadcast, only needed when the hart operates as a receiver

2.29.7. Workaround for Type G

If a TensorRecv, TensorReduce, or TensorBroadcast is followed by a TensorQuant on the same hart, the bug can be triggered when the last register written by the TensorReduce/TensorRecv/TensorBroadcast matches the second register read by the TensorQuant. Furthermore, if a TensorRecv, TensorReduce, or TensorBroadcast is followed by a TensorFMA32, TensorFMA16A32, or TensorIMA8A32 on the same hart, the bug can be triggered when the last register written by the TensorReduce/TensorRecv/TensorBroadcast is `f0`, which is the first register read by the TensorFMA32/TensorFMA16A32/TensorIMA8A32 operations. Specifically, the requirements to hit the issue are:

- For the TensorReduce or TensorBroadcast, the hart must be operating as a receiver
 - NOTE: The hart for a TensorRecv is always a receiver
- The TensorReduce/TensorRecv/TensorBroadcast must be performing an operation (`COUNT > 0`)

- The TensorFMA32/TensorFMA16A32/TensorIMA8A32 must not be masking row0 (i.e. MSK == 0 || tensor_mask[0] == 1)
- The final register written by the TensorRecv/TensorReduce/TensorBroadcast must be f0, the first register read by the subsequent TensorFMA32/TensorFMA16A32/TensorIMA8A32 (i.e. (FREG+COUNT-1)%32 == 0)
- The TensorQuant must be doing at least one transformation (i.e. TXF0 != LAST)
- The TensorQuant must be reading at least two registers from the register file (i.e. AROWS>0 || ACOLS >= 2)
- The final register written by the TensorReduce/TensorRecv/TensorBroadcast must match the second register read by the subsequent TensorQuant
 - NOTE: There is no issue with the first register read by the TensorQuant since that value bypasses the register file in the pipeline
 - The last register written by the TensorReduce/TensorRecv/TensorBroadcast is (FREG+COUNT-1)%32
 - The second register read by TensorQuant is as follows:

```

if(ACOLS >= 2 && AROWS == 0) {
    // This is the case where 1 row takes 2 VPU registers
    // and we are only processing 1 row
    2nd Register = (START+1)%32
} elseif(AROWS > 0) {
    // There are two cases for this:
    // 1) Each row takes up 2 VPU registers and
    //    we are processing more than 1 row
    // 2) Each row takes up 1 VPU register and
    //    we are processing more than 1 row
    2nd Register = (START+2)%32
} else {
    // In the case where 1 row takes 1 VPU register and
    // we are only processing 1 row, the bug is not triggered
}

```

The workaround for this is basically the same as that for Type F above:

1	<TensorOp>	# TensorReduce/TensorRecv/TensorBroadcast
2	<non-vpufp-inst>	# 0 or more instructions, none of which are Tensor Ops or FP instructions
3	fmv.x.w x0, f0	# This instruction is inserted to force the VPU pipeline to writeback its final result
4	<TensorQuant>	# This TensorQuant will read the correct results from the VPURF

Fix Status:

Not fixed.

2.30. Reset assertion may induce logic errors if not all the resets are asserted

ID: RTLMIN-6633 <https://esperantotech.atlassian.net/browse/RTLMIN-6633>

Owner: Xavier Revés

Block: Minion Shire

Description:

This is a description of a potential problem reported by a Reset Domain Crossing analysis tool that may not exist in reality. The purpose of this description is to provide a warning about the possibility of this issue in case any problems occur and to recommend possible workarounds.

There are multiple reset signals in the Minion Shire that are able to perform a warm reset to different parts of the Shire. When one of these parts is being reset but not the adjacent areas that have interfaces with it, there is some possibility that glitches at the interfaces will occur, which may result in unwanted triggers in the non-reset areas.

While it is unlikely to have a relevant signal (e.g., a “valid” signal) with a long glitch when the interfaces are already idle before the reset, the possibility must be taken into account.

Two main areas should be considered: 1) the Neighborhoods, which can be dynamically powered off (if the PCB design allows for this) and thus have a separate reset signal and 2) the Debug logic, which also has a separate reset.

Impact:

An interface may hang after reset if the glitch confuses the non-reset area.

Workaround:

The main recommendation would be to not apply a reset to only part of the system. However, if this cannot be avoided and a reset of only one area is strictly necessary, the reset should be performed at the lowest possible frequency. This way, the glitch will have time to recover before the following clock edge.

Notice that within the Minion Shire, all the areas (e.g., Shire Cache, Neighborhood) except the NoC are actually using the same clock, so they are synchronized. This means that it would be possible to sufficiently increase the clock period to be safe against any glitch while resetting a portion of the Shire.

Fix Status:

Not fixed.

3. Maxion and IOShire

3.1. Accessing a certain undefined address space (0x10) may hang the SP

ID: RTLMIN-4472 <https://esperantotech.atlassian.net/browse/RTLMIN-4472>

Owner: Lu Wan

Block: IOShire

Description:

If the service processor (SP) accidentally accesses an undefined address space, certain address spaces may not reply with responses (specifically 0x10 in the case of this bug). As a result, the SP will hang.

Impact:

The impact is minor. Our typical firmware usage knows where the memory holes are. In the case of normal/correct program usage, accessing those undefined memory ranges are invalid operations. Therefore, this only happens when there is a programming error. SW can easily work around this issue.

Workaround:

SW knows that accessing the undefined memory range is an invalid operation. A correctly written program would never generate such an access.

Fix Status:

Postponed.

3.2. Maxion return event counters miss counting some returns

ID: RTLMAX-342 <https://esperantotech.atlassian.net/browse/RTLMAX-342>

Owner: Eric Hao

Block: IOShire

Description:

The Maxion event counters for mispredicted returns and resolved returns may miss counting some instances of these returns. These event counters will not count returns that either:

1. Use x5 for its source 1 operand
2. Do not use x0 for its destination register.

Impact:

The impact is minor. The two cited event counters may undercount the returns in the profiled SW.

Workaround:

None.

Fix Status:

Postponed.

3.3. Maxion PTW requests are not checked for access violations

ID: RTLMAX-368 <https://esperantotech.atlassian.net/browse/RTLMAX-368>

Owner: Eric Hao

Block: IOShire

Description:

Requests generated by the hardware page table walker (PTW) are not checked for access violations because they bypass the TLB.

Impact:

This erratum opens up a small security hole. In theory, the PTW should never make an illegal access. However, if the OS inadvertently or maliciously fills the TLB with bad page table addresses, the PTW may end up accessing memory space for which it does not have access permission, including machine mode space.

Workaround:

None.

Fix Status:

Postponed.

3.4. Accesses to address-map holes in the Maxion subsystem result in undefined behavior

ID: RTLMAX-412 <https://esperantotech.atlassian.net/browse/RTLMAX-412>

Owner: Eric Hao

Block: IOShire

Description:

Accesses by the service processor or Minions to an undefined address within the Maxion address space will result in undefined behavior.

Impact:

This erratum opens up a small security hole. Our system firmware should not be making accesses to undefined addresses. However, if the FW inadvertently or maliciously accesses an undefined address, the resulting behavior may compromise the system in unexpected ways.

Workaround:

No workaround needed. A correctly written program should never access an undefined address.

Fix Status:

Postponed.

3.5. Maxion may raise speculative interrupts for bus errors on data accesses

ID: RTLMAX-485 <https://esperantotech.atlassian.net/browse/RTLMAX-485>

Owner: Eric Hao

Block: IOShire

Description:

Upon receiving a bus error for a data access, Maxion will immediately signal a bus error. Because data accesses are sent out speculatively, a speculative data access (i.e., a data access that was not requested by SW) may result in a bus error interrupt.

Impact:

Speculative interrupts due to bus errors may prevent correctly written programs from executing.

Workaround:

Workaround consists of two steps:

1. Reprogram the NOC so that it will pass on all accesses to DRAM space from maxion (i.e., it will not return an error response for accesses to illegal DRAM addresses)
2. PMP is programmed to restrict access to only legal DRAM space addresses

With this workaround, speculative accesses to illegal addresses will not be flagged with a data bus error. Non-speculative accesses to illegal addresses will be flagged with an access error exception that is raised by the PMP.

Fix Status:

Postponed.

3.6. Maxion debug single step functionality requires serial execution mode

ID: RTLMAX-494 <https://esperantotech.atlassian.net/browse/RTLMAX-494>

Owner: Eric Hao

Block: IOShire

Description:

To use the Maxion debug single step feature, the Maxion must be booted with serialize all defeature mode enabled.

Impact:

The impact is minor. This erratum affects only SW debuggers that wish to use the single step feature.

Workaround:

Boot Maxion with serialize all defeature mode enabled.

Fix Status:

Postponed.

3.7. Maxion debug status monitor bus valids are sometimes incorrect

ID: RTLMIN-5437 <https://esperantotech.atlassian.net/browse/RTLMIN-5437>

Owner: Eric Hao

Block: IOShire

Description:

The valids for the buses monitored by the Maxion debug status monitor remain asserted even when the status monitor's GPIO debug enable field is deasserted. However, deassertion of the debug enable field will result in incorrect data being driven on those buses. As a result, the status monitor may end up reporting status on invalid data.

Impact:

The impact is minor. This issue affects only the Maxion debug status monitor and can be easily worked around.

Workaround:

Assert the GPIO debug enable field before the status monitor is configured to monitor any of the data on its input buses.

Fix Status:

Postponed.

3.8. Maxion MISA.X value is incorrect

ID: RTLMAX-496 <https://esperantotech.atlassian.net/browse/RTLMAX-496>

Owner: Eric Hao

Block: IOShire

Description:

The MISA (machine ISA) register's X bit indicates whether the machine has any non-standard extensions. Maxion's MISA.X value is incorrectly set to 0. Since Maxion contains non-standard extensions, the MISA.X value should be set to 1.

Impact:

The impact is minor. This bit is used by SW only and it is not clear if any existing SW actually reads this bit.

Workaround:

None needed.

Fix Status:

Postponed.

3.9. UltraSoC ME reports incorrect GPIO parameters in discovery response

ID: RTLMIN-5831 <https://esperantotech.atlassian.net/browse/RTLMIN-5831>

Owner: Eric Hao

Block: IOShire

Description:

The discovery response sent by the Ultrasoc message engine reports that its GPIO bus width is 1 instead of the actual width of 26.

Impact:

The impact is minor. The discovery response lists all the message engine attribute values. The response is used to construct an XML file that describes the Ultrasoc components. This XML file is used by SW. We can correct the incorrect GPIO bus width in the XML file. Furthermore, the GPIO functionality is not affected by this bug. All 26 bits are driven correctly.

Workaround:

Manually correct the GPIO bus width in the XML file.

Fix Status:

Postponed.

3.10. SPIO UltraSoC BPAM module cannot read PLL/CRU registers

ID: RTLMIN-6174 <https://esperantotech.atlassian.net/browse/RTLMIN-6174>

Owner: Eric Hao

Block: IOShire

Description:

The UltraSoC BPAM module in the SPIO cannot receive read/write responses from the PLL/CRU blocks due to a bug in the APB response logic. As a result, the BPAM module cannot read registers in those blocks and cannot receive a write error response.

Impact:

The major impact is that the BPAM module cannot read the PLL/CRU registers. It can write those registers but will not be able to receive an error response for any of those writes. This bug does not affect the BPAM module in the Minion Shire.

Because the BPAM module was meant as a backup to the UltraSoC DMA module to allow accesses in the case that the SPIO NoC is broken, the impact to debug functionality is minimal.

Workaround:

These registers can be accessed by one of the following:

- Service processor
- UltraSoc DMA module
- Scandump

Fix Status:

Postponed.

3.11. ETL2AXI bridge does not handle illegal requests from the SP

ID: RTLMIN-5960 <https://esperantotech.atlassian.net/browse/RTLMIN-5960>
RTLMIN-6134 <https://esperantotech.atlassian.net/browse/RTLMIN-6134>

Owner: Eric Hao

Block: IOShire

Description:

If the ETL2AXI bridge receives an illegal request from the SP, it may hang, which in turn would hang the SP.

Impact:

The impact is minor. As long as SW avoids issuing outlawed requests, this erratum will have no effect. This may make SW development a bit more challenging, but the final working version of the SP SW will be unaffected.

Workaround:

Do not use the requests outlawed in the PRM (section 2.4).

Fix Status:

Postponed.

3.12. SP allows cache operations to be illegally performed on uncacheable space

ID: RTLMIN-6148 <https://esperantotech.atlassian.net/browse/RTLMIN-6148>

Owner: Eric Hao

Block: IOShire

Description:

The SP does not raise an access fault exception for cache operations (prefetch and lock) applied to addresses in DRAM uncacheable space. As a result, the SP will inappropriately allow such operations to be performed. This may result in data corruption.

Impact:

No impact if the SW workaround is used. Otherwise, data may be corrupted.

Workaround:

SW should never attempt to execute a cache operation on an address in DRAM uncacheable space.

Fix Status:

Postponed.

3.13. sms_sft_ready/fail interrupts do not work before PLLs are configured

ID: RTLMIN-6232 <https://esperantotech.atlassian.net/browse/RTLMIN-6232>

Owner: Eric Hao

Block: IOShire

Description:

Due to clock domain crossing issues, the sms_sft_ready/fail interrupts may not work. Interrupt requests that are sent from these sources before the PLLs are configured may be missed by the SPIO PLIC.

Impact:

No impact. The only time the SMS logic would raise these interrupts is when the SP is held in reset. As a result, during the window in which these interrupts could be raised, the SP would not yet have enabled them.

Workaround:

SW should never enable these interrupts.

Fix Status:

Postponed.

3.14. Perstn assert/deassert interrupts do not work before PLLs are configured

ID: RTLMIN-6232 <https://esperantotech.atlassian.net/browse/RTLMIN-6232>

Owner: Eric Hao

Block: IOShire

Description:

Due to clock domain crossing issues, the perstn assertion/deassertion interrupts may not work. Interrupt requests that are sent from these sources before the PLLs are configured may be missed by the SPIO PLIC.

Impact:

No impact. The SP does not pay attention to these interrupts until after the PLLs are configured. If we should ever run this part with ios_clk running at the same or lower frequency as ios_clk_apb, then the perstn interrupts may be missed by the SPIO PLIC. This operating mode should never happen in the product.

Workaround:

None needed.

Fix Status:

Postponed.

3.15. Scandump does not work due to incorrectly connected stop clock module

ID: RTLMIN-5976 <https://esperantotech.atlassian.net/browse/RTLMIN-5976>

Owner: Eric Hao

Block: IOShire

Description:

The TE/SE port of a stop clock module in the CRU was incorrectly connected to dft_clk_gate_en; it should have been connected to the constant 0. As a result, if dft_clk_gate_en is asserted during the stop clock phase of scandump, this clock will not stop.

Impact:

No impact if the SW workaround is used. Otherwise, the data from the scandump will be corrupted.

Workaround:

Set dft_clk_gate_en to 0 (either by driving it from the pad or setting it via the TDR) during the clock stop phase of the scandump. When you are ready to shift the scandump data, dft_clk_gate_en must be set back to 1.

Fix Status:

Postponed.

3.16. DFT does not have appropriate control of the eMMC PHY clock

ID: RTLMIN-6158 <https://esperantotech.atlassian.net/browse/RTLMIN-6158>

Owner: Eric Hao

Block: IOShire

Description:

Since the wrong clock was connected to the DFT clock mux, we cannot drive a 200 MHz clock into the eMMC PHY during test mode. 200 MHz is the operating frequency for this IP.

Impact:

Some flops cannot be captured by DFT while DFT is running at its 200 MHz operating frequency, but they can be captured at the slower 100 MHz speed. This will affect about 650 flops. The file listing the affected flops is attached to the Jira issue.

DFT can still test them for stuck-at faults.

Workaround:

None.

Fix Status:

Postponed.

3.17. Vault Scan Chain cannot be isolated

ID: RTLMIN-5502 <https://esperantotech.atlassian.net/browse/RTLMIN-5502>

Owner: Eric Hao

Block: IOShire

Description:

The vault was supposed to be on its own scan chain so that access to it could be permanently disabled, guaranteeing that the private keys stored in the vault (in particular the customer's private keys) could not be accessed by anyone, including Esperanto. Unfortunately, the scan chain was not isolated and this security feature will not be provided.

Impact:

Customer private keys installed in the vault will be visible to Esperanto if they return parts to us.

Workaround:

Inform customers about the absence of this security feature. Propose that customers utilize non-production keys during product development so that when they send us devices to debug, they are not worried about their secrets being exposed. Once their system goes into production, they should plan to either revoke their secrets or avoid sending us their devices.

Fix Status:

Postponed.

3.18. Prefetch CacheOp will hang Maxion

ID: RTLMAX-504 <https://esperantotech.atlassian.net/browse/RTLMAX-504>

Owner: Stephen Lee

Block: Maxion MAU

Description:

Using the Prefetch CacheOp will hang the Maxion CPU. Normally, the DCache will send an acknowledge signal back to the load buffer when the data is ready, but the logic does not take into account the prefetch operation. The load buffer will wait forever for an acknowledgement of data and will hang, triggering a retire watchdog timer exception.

Impact:

Software will hang randomly if Prefetch CacheOp is used. There is no impact if this instruction is not used.

Workaround:

Do not use Prefetch CacheOp.

Fix Status:

Not fixed. Future Maxion revisions will likely implement RISC-V Standard Cache Ops instead.

3.19. CSR tdata1 match field reset value is not 0, as defined in the RISC-V Debug Specification version 0.13

ID: RTLMAX-509 <https://esperantotech.atlassian.net/browse/RTLMAX-509>

Owner: Stephen Lee

Block: Maxion CSR file

Description:

Only bits [1:0] in the Maxion tdata1 CSR are writable; the other match field bits are tied to 0. However, these writable bits have to be initialized to 0 at reset, as defined in the Debug Specification. If this is not done, match[1:0] will be random after reset.

Impact:

Debug software that assumes the match field is 0 automatically after reset will not function properly.

Workaround:

Software needs to initialize tdata1 by writing 0 before running any code involving debug logic (e.g., writing the trigger CSRs to detect what features the trigger module supports).

In addition, it is also strongly recommended that tdata2 be initialized to 0.

Fix Status:

Not fixed.

3.20. To be RISC-V compliant, tdata3 should be implemented as a register instead of trapping as an illegal instruction

ID: RTLMAX-516 <https://esperantotech.atlassian.net/browse/RTLMAX-516>

Owner: Stephen Lee

Block: Maxion CSR file

Description:

Maxion follows the RISC-V Debug Specification v0.13-DRAFT dated June 2019. This spec does not state that tdata3 is optional. Maxion does not implement tdata3 as a register but will trap it as an illegal instruction when it is accessed. tdata3 usage is never defined in v0.13, so having it as a register is redundant. In v.10, tdata3 has been rectified as optional, so the current implementation is future proof.

Impact:

There is no impact, software should never use tdata3 in the Maxion environment.

Workaround:

Software should never access tdata3.

Fix Status:

Not fixed. Won't be fixed in future revisions.

3.21. TileLink to AXI signals BREADY and RREADY are not initialized at reset

ID: RTLMAX-526 <https://esperantotech.atlassian.net/browse/RTLMAX-526>

Owner: Stephen Lee

Block: Maxion Neighborhood TileLink to AXI interface

Description:

The value of BREADY and RREADY are unknown after reset. These are handshake signals TileLink uses to notify the interface that it is ready to accept a write or read response. Once Maxion initiates one of these requests through TileLink, the ready signals will be set to the proper value. Since Maxion needs to start a request before it can communicate its readiness for a response, an initial state of these ready signals right after reset is not necessary.

Impact:

There is no impact, it only creates a lot of X-assertion in the DV environment.

Workaround:

Software can initialize these READY signals with a few code sequences at init:

1. Load and Store to a safe MMIO address
2. Load and modify a cache line, then flush it

Fix Status:

Postponed.

3.22. Flush Op completion does not wait for ReleaseAck from TieLink

ID: RTLMAX-525 <https://esperantotech.atlassian.net/browse/RTLMAX-525>

Owner: Stephen Lee

Block: Maxion Data Cache

Description:

Flush Ops {flushvar, flushvaw, flushall} in Maxion are non-standard RISC-V instructions; they are implemented using CSR write instructions to custom CSR indexes. These instructions break the TileLink protocol since they do not observe the ReleaseAck response from TileLink. Depending on the TileLink implementation, the Maxion core may read back stale data after the flush, or it may respond to another core's probe too early with incorrect coherency information.

Impact:

Since these are non-standard RISC-V instructions, standard software applications will not use these and will not be affected. However, debug, DV diagnostics, and in-house software that uses these operations may get incorrect results.

Workaround:

For flushvar and flushvaw, the following code sequence should be used to guarantee that the flush operation will be completed:

1. Flushva addr A
2. Load addr A (serialized by the flush above due to matching address)
3. Fence (only completes after load returns)
4. Flushva addr A (invalidates the line)

For flushall, the following code sequence should be used to guarantee that the flush operation will be completed:

1. Flushall
2. Load addr A where A is any address that maps to shire bank 0
3. Load addr B where B is any address that maps to shire bank 1
4. Load addr C where C is any address that maps to shire bank 2
5. Load addr D where D is any address that maps to shire bank 3
6. Fence
7. Flushva addr A
8. Flushva addr B
9. Flushva addr C

10. Flushva addr D

Fix Status:

Not fixed. Future Maxion revisions will likely implement RISC-V Standard CacheOps instead.

3.23. sfence generates a non-committed instruction fetch using mtvec

ID: RTLMAX-534 <https://esperantotech.atlassian.net/browse/RTLMAX-534>

Owner: Stephen Lee

Block: Maxion Execution Unit

Description:

sfence will create an instruction fetch to the address stored in mtvec. Even though this fetch is speculative, it is not marked by the fetch unit. The mtvec CSr is reset to 0. Hence, if mtvec is not properly initialized by software, sfence will generate a fetch to 0x0, which is MMIO space.

Impact:

Very Low. The initialization code will normally set mtvec to a safe cacheable space long before any software executes any sfence instruction.

Accessing MMIO space speculatively does have serious undesirable side-effects if the workaround is not performed.

Workaround:

Initialize mtvec with an address pointing to a safe non-MMIO space after reset before executing the first sfence.

Fix Status:

Postponed.

3.24. mtval CSR is written with an improperly sign-extended address

ID: RTLMAX-617 <https://esperantotech.atlassian.net/browse/RTLMAX-617>

Owner: Stephen Lee

Block: Maxion Execution Unit

Description:

mtval CSR is defined as a 64-bit register in the RISC-V privileged spec. In the Maxion implementation, it is a 49-bit register. When this CSR is read, bit[48] will be sign-extended to generate the full 64-bit data.

Maxion implements 48-bit virtual address and 40-bit physical address.

In certain corner cases within an exception, the mtval CSR can be written with an address that is not properly sign-extended, i.e. bit[48] is not equal to bit[47].

Note: the stval CSR has the same issue. See erratum 3.32

Impact:

Low. The exception handler can fix the sign-extension of the bad address before it is used.

Workaround:

Here's the code to fix the sign-extension.
`mtval = (ulong)((long) mtval << 16 >> 16);`

By left-shifting 16 bits and followed by arithmetic right-shifting 16 bits, the mtval will guarantee to be properly sign-extended.

Please beware that this workaround will also make an illegal address legal and suppress a fault that should have happened. For a complete workaround, it is recommended that the code should ignore mtval, parse the instruction in the traphandler, locate the register and compute the address.

Fix Status:

Postponed

3.25. Fdiv.s and fsqrt.s with SNaN operands do not set Not Valid flag (fflags[NV])

ID: RTLMAX-505 <https://esperantotech.atlassian.net/browse/RTLMAX-505>

Owner: Eric Hao

Block: Maxion Floating Point Unit

Description:

When passed a NaN operand, the fdiv.s and fsqrt.s do not set the Not Valid flag (fflags[NV]) as required by the IEEE 754 standard. Note: this issue does not occur with the double precision variants of these opcodes.

Impact:

Unknown. This is an obscure case that may not occur in most FP code.

Workaround:

None.

Fix Status:

Postponed..

3.26. Maxion PLL registers cannot be accessed when the Maxion Uncore Clock is running at target frequency

ID: RTLMAX-560 <https://esperantotech.atlassian.net/browse/RTLMAX-560>

Owner: Eric Hao

Block: Maxion CRU

Description:

Access logic to the Maxion core and uncore PLL registers is driven by the uncore clock. This logic is designed to work up to 200MHz. However, when the uncore clock is driven by the uncore PLL, its frequency will be the frequency of the Maxion uncore PLL, which has a target frequency of 1GHz. This frequency exceeds the limit of 200 MHz for the register access. As a result, these registers cannot be accessed when the Maxion uncore clock is driven by the uncore PLL when it is running at its target frequency.

Impact:

Low - in normal operating circumstances, these PLL registers do not need to be accessed once the uncore PLL is running at its target frequency. The only situation that might warrant access to these registers is if we were attempting to debug an issue that involved the Maxion PLLs.

Workaround:

To access the Maxion core and uncore PLL registers after the uncore PLL has been configured to run at target frequency, switch the uncore PLL source back to the reference clock. The uncore PLL will then be running at a low enough frequency (100 MHz) such that those registers can be safely accessed.

Fix Status:

Postponed.

3.27. Maxion feature control bit to enable clearing of load reservation on voluntary eviction does not work

ID: RTLMAX-531 <https://esperantotech.atlassian.net/browse/RTLMAX-531>

Owner: Eric Hao

Block: Maxion LSU

Description:

By default, Maxion does not clear a load reservation if the reserved cache line is evicted from the cache due to capacity reasons. In theory, this means that in this situation, a store conditional can still succeed despite missing in the L1 dcache. In addition, stores from other Maxion cores will be blocked until this reservation is released. The Maxion feature control unit provides a bit that enables clearing the reservation in this situation. This bit, Irsc-clearorel, is described in section 6.6.1.5 of the [Maxion Shire MAS](#).

This bit does not work. The reservation is not cleared due to a capacity eviction.

Impact:

Low - while operating at the target frequencies for the core (1.5GHz) and the uncore (1GHz), a capacity eviction of a reserved line will always result in the reservation being effectively canceled regardless of the setting of the feature control bit. This is because at the target frequencies, the number of cycles needed to service a cache miss exceeds that of the timeout count for the reservation. This guarantees that a subsequent store conditional will always fail because the reservation is guaranteed to be cleared due to the timeout.

Workaround:

Run at target core and uncore frequencies.

Fix Status:

Postponed.

3.28. IOShire SPIO PLL0 Output Frequency Beats with the SPI ssi_clk

ID: SW-11072 <https://esperantotech.atlassian.net/browse/SW-11072>
SW-11416 <https://esperantotech.atlassian.net/browse/SW-11416>

Owner: Daniel Yau

Block: IOShire SPIO

Description:

The issue of the boot sequence in the Boot ROM not completing in some chips is reported in JIRA SW-11072. The issue has been identified to be related to the frequency relationship between the SPIO PLL0 and the SPIO PLL1. The ROM has configured the PLL0 frequency to 1000 MHz and the PLL1 frequency to 2000 MHz. The derived clock clk_ios from PLL0 becomes 500 MHz and the derived clock clk_500mhz from PLL1 is also 500 MHz. The clk_500mhz is used for the SPI ssi_clk. Both clocks are used in the SPI IP, in two separate clock domains. There are Asynchronous FIFOs between the two clock domains. While both clocks in an Asynchronous FIFO have the same frequency, there is a probability to substantially reduce the Mean Time Before Failure (MTBF) of the synchronizers in the FIFO. When the SPI has a failure due to this issue, the ROM code is unable to read the boot codes stored in the on-package flash memory since it is connected to an SPI controller. As a result, the boot sequence fails.

Impact:

High - Failing to boot causes system failure.

Workaround:

The workaround is to reduce the PLL0 frequency by 0.5%. The details can be found in SW-11416. The mechanism to workaround this issue is to use the eFuse words allocated to PLL Configuration Register Value Override. The ROM code detects that such words are valid and the words are used to override the default values hardcoded in the ROM. The exact override values and the PLL Configuration Registers impacted can be found in the [ET-SOC E_Fuse_Allocation_v2.5_with_SP_and_Vault_info](#) spreadsheet. See rows 174, 175, and 176 in banks 43 and 44. The version of the file is May 17, 2002, modified by Raj Lam.

Fix Status:

Not fixed in silicon.

3.29. PTW accesses are not checked for access violations

ID: RTLMAX-368 <https://esperantotech.atlassian.net/browse/RTLMAX-368>

Owner: Eric Hao

Block: Maxion LSU

Description:

The memory accesses generated by the page table walker are passed directly to the L1 DCache and do not pass through the TLB. As a result, no permission checks are performed on these accesses.

Impact:

Maxion has a security hole with respect to the PTW running in S-mode. If the PTW has a bug or is maliciously compromised, Maxion will not stop it from making illegal accesses.

Workaround:

None.

Fix Status:

Postponed.

3.30. Maxion does not prevent unprivileged access to its ESRs

ID: RTLMAX-519 <https://esperantotech.atlassian.net/browse/RTLMAX-519>

Owner: Eric Hao

Block: Maxion LSU

Description:

Maxion does not prevent accesses to ESRs for which they do not have the requisite privilege.

Impact:

Maxion has a security hole in which its programs can access ESRs for which they do not have the requisite privilege.

Workaround:

Program PMPs and page tables to prevent such illegal accesses from occurring. This workaround is not bulletproof due to erratum 2.31. Illegal read accesses to IO addresses with read side effects will still trigger the read side effects despite having an access fault exception raised for that read instruction.

Fix Status:

Postponed.

3.31. Maxion PMPs allow barred accesses to leave core

ID: RTLMAX-519 <https://esperantotech.atlassian.net/browse/RTLMAX-532>

Owner: Eric Hao

Block: Maxion LSU

Description:

Although the Maxion PMP logic will raise an access fault exception for accesses to barred addresses, it will still allow accesses that are reads to leave the core. As a result, the read is done in a micro-architectural sense despite its result not being committed architecturally. This is a problem for reads to IO addresses that have a side effect. PMPs will not be able to prevent such illegal accesses from triggering the associated side effect.

Impact:

Maxion has a security hole in which IO addresses with read side effects cannot be fully protected by PMPs.

Workaround:

None

Fix Status:

Postponed.

3.32. stval CSR is written with an improperly sign-extended address

ID: RTLMAX-631 <https://esperantotech.atlassian.net/browse/RTLMAX-631>

Owner: Eric Hao

Block: Maxion Execution Unit

Description:

stval CSR is defined as a 64-bit register in the RISC-V privileged spec. In the Maxion implementation, it is a 49-bit register. When this CSR is read, bit[48] will be sign-extended to generate the full 64-bit data.

Maxion implements 48-bit virtual address and 40-bit physical address.

In certain corner cases within an exception, the stval CSR can be written with an address that is not properly sign-extended, i.e. bit[48] is not equal to bit[47].

Note: the mtval CSR has the same issue. See erratum 3.24

Impact:

Low. The exception handler can fix the sign-extension of the bad address before it is used.

Workaround:

Here's the code to fix the sign-extension.

```
stval = (ulong)((long) mvtal << 16 >> 16);
```

By left-shifting 16 bits and followed by arithmetic right-shifting 16 bits, the mtval will guarantee to be properly sign-extended.

Please beware that this workaround will also make an illegal address legal and suppress a fault that should have happened. For a complete workaround, it is recommended that the code should ignore stval, parse the instruction in the traphandler, locate the register and compute the address.

Fix Status:

Postponed

3.33. Stores to a DCache E state line may be dropped

ID: RTLMAX-635 <https://esperantotech.atlassian.net/browse/RTLMAX-635>

Owner: Eric Hao

Block: Maxion LSU

Description:

A store to a line in E state in the DCache may be dropped if the right sequence of events occurs around the time of the store. One of those events is a probe to a line in the same set as the line the store is targeting.

In ET-SoC-1, the only way a line can be installed in E state is by executing a LR instruction. The LR brings the line into the DCache. If no LR instructions are executed, then the DCache will not have any lines in E state and this bug will not occur.

Impact:

Medium. There is no sure fire way to avoid this issue except for not using the LR instruction.

Workaround:

Don't use LR instructions.

Fix Status:

Postponed

3.34. PTW may corrupt PMP access check for subsequent memory operation

ID: RTLMAX-636 <https://esperantotech.atlassian.net/browse/RTLMAX-636>

Owner: Eric Hao

Block: Maxion LSU

Description:

If the Maxion has a TLB miss and the PTW returns the missing PTE while the LSU is executing either:

1. A memory operation when virtual memory is disabled
2. A memory operation that maps to a physical memory page that is covered by more than one PMP entry

then that memory operation will not have the proper PMP checks applied to it. This may result in either an access exception being raised when it shouldn't or an access exception not being raised when it should.

Impact:

Medium.

Workaround:

The SW fix has two components to prevent the bug:

1. Require that PMPs must be programmed such that all pages within the TLB are guaranteed to be homogenous. At a minimum, this means that all PMP regions must have a size that is a multiple of 4 KB and an alignment that starts on a 4 KB boundary. If the TLB has superpages, we will need more requirements to guarantee that those pages are homogeneous too.
2. Whenever the core transitions from virtual memory being enabled to being disabled (e.g., when we enter M mode), SW must execute a fence before any memory operation is executed. The fence will stall the pipeline until the page table walk has completed.

Fix Status:

Postponed

3.35. Minstret is off by one after a write

ID: RTLMAX-637 <https://esperantotech.atlassian.net/browse/RTLMAX-637>

Owner: Eric Hao

Block: Maxion EXU

Description:

After a write to the minstret CSR, minstret should hold the value written. However, minstret counts the retirement of the CSR write. As a result, minstret will hold the value written + 1.

Impact:

Low.

Workaround:

None. The minstret count will always be off by one after a write to it. SW won't care about this small inaccuracy in the instruction retired count.

Fix Status:

Postponed

3.36. Cache Access Bus Errors Record Incorrect MCAUSE Value

ID: [GPD-2512](#)

Owner: Eric Hao

Block: Maxion EXU

Description:

When a cache access receives an error response, the MCE unit, if configured appropriately, will raise an interrupt. For this interrupt, the MCAUSE CSR should be set to 0x8000_0000_0080_0000 indicating a custom interrupt. Instead, the current implementation sets MCAUSE to 0x8000_0000_0000_0080 which incorrectly indicates a machine timer interrupt.

Impact:

Low.

Workaround:

The SW interrupt handler will have to check the MCE Cause CSR to determine if the interrupt is a timer interrupt or a bus error interrupt. See the [Maxion Tile MAS](#) for more information about the MCE Cause CSR.

Fix Status:

Postponed

4. Shire Cache

4.1. Neigh request hangs if shire cache full of l3_slave requests to same SCP address

ID: RTLMIN-6207 <https://esperantotech.atlassian.net/browse/RTLMIN-6207>

Owner: Kristie Veith

Block: Shire Cache

Description:

The shire cache prioritizes l3_slave requests over neighborhood requests. If there are too many l3 requests there may be no resources for neighborhood requests. l3_yield_priority allows priority and resources to be given to neighborhood requests. However, if the neighborhood and the mesh are both accessing the same SCP request it is possible for the neighborhood request to continually be squashed and the request could still not progress even with l3_yield_priority set to non-zero.

Impact:

Low. It would have to be a pretty consistent and repeating pattern for the neighborhood to continually be squashed and re-queued.

Workaround:

Slow down the polling rate from other shires that are waiting for the local neighborhood to complete whatever task it is trying to complete.

Fix Status:

Postponed.

4.2. L3_yield priority does not have sub-bank granularity

ID: RTLMIN-6214 <https://esperantotech.atlassian.net/browse/RTLMIN-6214>

Owner: Kristie Veith

Block: Shire Cache

Description:

The shire cache prioritizes l3_slave requests over neighborhood requests. If there are too many l3 requests there may be no resources for neighborhood requests. l3_yield_priority allows priority and resources to be given to neighborhood requests. However the priority does not have sub-bank granularity.

It is possible for l3_slave requests to over use one of the sub-banks and then that sub-bank is busy when it's the neighborhood's turn to make a request due to l3_yield_priority. The neighborhood request uses a different sub-bank which satisfies l3_yield_priority. As a result a request to the over used sub-bank from a neighborhood may not make progress despite l3_yield_priority being non-zero.

Impact:

Low. It would have to be a pretty consistent and repeating pattern for the neighborhood request to a particular sub-bank to be skipped over and over.

Workaround:

Access patterns should be spread pretty evenly across sub-banks.

Fix Status:

Postponed.

4.3. Performance Monitor Cycle Counter is counting incorrectly when there is no outstanding requests in Shire Cache Bank

ID: RTLMIN-6235 <https://esperantotech.atlassian.net/browse/RTLMIN-6235>

Owner: Bill Lee

Block: Shire Cache

Description:

The Performance Monitor (PM) Cycle counter counts the number of cycles that it is enabled for. However when the Shire Cache Bank (SCB) has no outstanding requests to process, the gated clock to the Cycle counter is shut off which stops the Cycle counter, resulting in an undercounting of cycles that it is enabled for.

Impact:

Low. The severity of this issue is minor given that this problem only occurs with performance measurement and software workarounds are provided.

Workaround:

- 1) Disable the SCB clock gating with the ESR `esr_sc_clk_gate_disable[0]` set to 1. This will enable the free running clock to drive the PM Cycle counter. Implication is that the SCB will consume more power when doing performance measurement. However, the `esr_sc_clk_gate_disable` can be selectively set for the SCBs that are collecting performance data, hence reducing the impact of the power consumption.
- 2) Use an external Cycle counter like the one in the minions in conjunction with the PM other counters. Implication is that it will be a bit off from what the PM Cycle counter will measure if it has a free running clock.

Fix Status:

Postponed.

4.4. SCP Fill requests from SCP source address using self shire id does not translate outgoing shire_id to actual shire_id

ID: RTLMIN-6579 <https://esperantotech.atlassian.net/browse/RTLMIN-6579>

Owner: Kristie Veith

Block: Shire Cache

Description:

An Scp_Fill operation with a source fill address coming from the same shire's SCP region using the "self shire id" mapping (0xFF) translates the address field's "self shire id" mapping to scp_shire_id (i.e. actual shire id) but does not translate the source fill address "self shire id" mapping located in the data field. So the request going to the NOC over the to_L3 port has a Remote SCP request with a shire_id=0xFF which is not supported by the NOC

Impact:

Low. If SW uses the "self shire id" in the data field, it will receive an AXI Decode ERR and an interrupt to the SP PLIC. However, there is an easy SW workaround.

Workaround:

SW should not use "self shire id" for the source fill address located in the data field of the et-link command. For SCP Fill operations, SW should always use the actual shire id within the address of the data field.

Fix Status:

Postponed.

4.5. If 2 errors occur in the same cycle, one gets logged, but the multi bit does not get set.

ID: RTLMIN-6616 <https://esperantotech.atlassian.net/browse/RTLMIN-6616>

Owner: Kristie Veith

Block: Shire Cache

Description:

In the Shire Cache error logger if an error occurs it gets logged into esr registers. If a second error occurs, the multi bit gets set to indicate that there is a second error.

However, if two errors occur in the same cycle, the multi bit should get set, but does not.

Impact:

Low.

Workaround:

This is background info on errors which shouldn't be happening, much less two in the same cycle. If we miss one SW error, we'll see it again after fixing the first SW error. If the errors are ECC two bit errors, we don't really care if there is 1 or 2 of them, either way we restart.

Fix Status:

Postponed.

4.6. Remote SCP Write Arounds and Cache Evicts are not ordered against other Remote SCP actions to the same address.

ID: RTLMIN-6616 <https://esperantotech.atlassian.net/browse/RTLMIN-6617>

Owner: Kristie Veith

Block: Shire Cache

Description:

Write Arounds to remote SCP addresses are coalesced in the local L2 cache memory. To do that they were put into the "L2" address space. However, when that happened they ceased to be in the SCP address space anymore and so are no longer ordered against other remote SCP accesses.

The following sequence will fail:

```
Store old_data
WriteAround new_data
Cache evict the new_data
Ld should get new_data
```

However, the load gets the old_data because it can go out on the mesh without waiting for the WriteAround and cache evict.

Impact:

Low.

Workaround:

The workaround is to put a barrier after the Cache Evict. Or, more likely, use the CacheOp state machine that is intended to flush all WriteArounds that are still in the L2, and that does work.

Fix Status:

Postponed.

4.7. Evict from L2 of a remote SCP address sent to wrong NoC port if remote_l3_enable=0.

ID: RTLMIN-6634 <https://esperantotech.atlassian.net/browse/RTLMIN-6634>

Owner: Kristie Veith

Block: Shire Cache

Description:

Write Arounds to remote SCP addresses are coalesced in the local L2 cache memory. To do that they were put into the "L2" address space. Similarly, Evicts to remote SCP addresses are also put into the "L2" address space. However, when that happens, if remote_l3_enable=1, the evicted remote SCP data is sent to the to_sys noc port toward memory instead of to remote scp.

The following sequence will fail if remote_l3_enable=0.

```
Store old_data
WriteAround new_data
Cache evict the new_data
Ld should get new_data
```

The cache evict will get sent to main memory and be a decode error.

Impact:

Low.

Workaround:

Don't use write arounds to coalesce remote SCP data if remote_l3_enable=0.

Fix Status:

Postponed.

5. Memshire

5.1. Memshire East NoC voltage monitors are connected incorrectly

ID: PD-621 <https://esperantotech.atlassian.net/browse/PD-621>

Owner: Tom Berg

Block: Memshire East (Memshires 4-7)

Description:

The voltage monitors intended to measure the NoC voltage on Memshires 4 through 7 are connected to the Memshire base voltage instead.

Impact:

It is not possible to directly measure the NoC voltage on Memshires 4 through 7.

Workaround:

A close approximation of the NoC voltage at these Memshires can be determined by reading the NoC voltage from Minion Shires that are physically adjacent as follows:

<u>Memshire #</u>	<u>Adjacent Minion Shire (Virtual ID)</u>
4	5
5	13
6	6
7	14, 15

Fix Status:

Postponed.

5.2. Diagnostic poisoning DDR data requires SW workaround

ID: Synopsys STAR 9001389475

Owner: Tom Berg

Block: DDR Controller

Description:

This erratum comes from Synopsys STAR 9001389475: ECCPOISONPAT0/1/2 Registers Delayed Clock Crossing after sw_done_ack is asserted.

The DDR controller provides the ability to poison data in the DDR (i.e., write bad ECC), which can be used to test the hardware and software ECC error handling mechanisms. This erratum describes a problem with the poisoning mechanism.

An incorrect ECCPOISONPAT0/1/2 pattern is used when poisoning write data for the advanced ECC feature due to the delayed clock domain crossing of these registers compared to SWCTL.sw_done/SWSTAT.sw_done_ack handshaking.

As a result, the poisoning that occurs may not match that expected by the programmed ECCPOISONPAT0/1/2 (i.e., it may or may not see a correctable/uncorrectable error as expected).

Impact:

The severity of this issue is minor given that this problem only occurs with diagnostic functionality and a software workaround is provided.

Workaround:

As part of the software sequence outlined in "Table 2-85: Procedure to Write into the ECCPOISONPAT0/1/2 Register Indirectly" of the uMCTL2 Databook (v 3.50a), after step 4 has been completed, modify the sequence to wait 128 pclk cycles, and then execute "Step 5: Set SWCTL.sw_done to 1".

This will ensure that the clock domain crossing for the ECCPOISONPAT0/1/2 registers has completed.

Fix Status:

No plan to fix in silicon.

5.3. Changing DDR Controller frequencies requires SW workaround

ID: Synopsys STAR 9001511588

Owner: Tom Berg

Block: DDR Controller

Description:

This erratum comes from Synopsys STAR 9001511588: Issue with DFITMG0.dfi_tphy_wrlat/dfi_tphy_wrdata in frequency change procedure.

PHY interoperability compliance failure caused by interface timing/sequence/parameter/etc. Updating DFITMG0.dfi_tphy_wrlat or DFITMG0.dfi_tphy_wrdata as part of a frequency change procedure may cause metastability and/or glitches in the following signals:

*dfi_phyupd_ack
dfi_ctrlupd_req*

As a result, when changing the frequency on the DDR Controller, the provided software workaround must be followed.

Impact:

The severity of this issue is minor given that this problem only occurs when changing frequencies and a simple software workaround is provided.

Workaround:

After the clock frequency is changed, set the following registers prior to changing any other registers:

DFIUPD0.dis_auto_ctrlupd = 1'b1
Disable dfi_phyud_req in the PHY

After the timing registers are changed for the new frequency, set the following registers back to their original values, if appropriate:

DFIUPD0.dis_auto_ctrlupd = 1'b0
Re-enable dfi_phyud_req in the PHY

Fix Status:

No plan to fix in silicon.

5.4. Changing DDR Controller frequencies using a new PSTATE requires SW workaround

ID: Synopsys STAR 3256585

Owner: Tom Berg

Block: DDR Controller

Description:

This erratum comes from Synopsys STAR 3256585: When using multiple PSTATES, the PLL lock time may not be sufficient.

PLL calibration values for PSTATE P0 are used when attempting a fast relock for any PSTATE. These values may not be sufficient to guarantee relock in 2 us for PSTATES other than P0.

The DDR PHY can store up to 4 sets of training values, called PSTATES. This erratum describes a problem when switching from one PSTATE to another, which could occur when changing DDR frequencies.

Therefore, when changing frequencies with a new PSTATE, the provided software workaround must be followed.

Impact:

The severity of this issue is minor given that this problem only occurs when changing frequencies on the DDR and a simple software workaround is provided.

Workaround:

1. For systems that can tolerate a longer frequency change time, set the CSR Seq0bDly0 for each PSTATE for a 4.5 us delay prior to a frequency change or fast relock (in user customPostTrain).
2. This will increase the tinit_complete from 3 us to 11 us.
3. For systems that cannot accommodate the increase in tinit_complete or where it is preferable to keep the same frequency as the versions of Firmware and PIE currently in use (A-2017.05 or later), perform per-PSTATE calibration and store the results in software. CSRs (PIICtrl3) from these saved results should then be restored prior to a frequency change or fast relock.

Fix Status:

No plan to fix in silicon.

6. Pshire

6.1. PCIe Controller PCI_MSI_EXT_DATA_EN register write must be done by DW write accesses

ID: Synopsys STAR ID 3099649 FV-187 <https://esperantotech.atlassian.net/browse/FV-187>

Owner: Daniel Yau

Block: PCIe Controller within PCIe Subsystem

Description:

The error was reported by Synopsys STAR report as shown in the ID section. When a field in the register is written with a byte write, the write data is not written to the field. The write access is completed with the write data dropped.

The generated RTL codes in the controller have this bug such that the write access to this register must be done with a DW write.

Impact:

The write access to this register completed incorrectly can cause the intended MSI operation not producing expected results.

Workaround:

The FW write accesses to this register must use DW accesses.

Fix Status:

The FW has been reviewed and the workaround has been applied.

6.2. PCIe Controller unable to enter loopback state in special use case

ID: Synopsys STAR ID 3901714 RTLMIN-6588

<https://esperantotech.atlassian.net/browse/RTLMIN-6588>

Owner: Daniel Yau

Block: PCIe Controller within PCIe Subsystem

Description:

The error was reported by Synopsys STAR report as shown in the ID section. When the controller is a loopback slave in the Loopback.Entry state, operating at GEN3 data rate or higher, it cannot enter Loopback.Active state in a special use case.

When the error is exposed, the loopback test will fail.

The special use case exposes this error involves the following conditions:

1. The connection to the link partner has lane reversal.
2. The loopback test starts after a speed change to GEN3 or higher data rate.
3. The speed change follows the following LTSSM state transitions: L0/Recovery/Hot Reset/Detect/Polling/Configuration.Linkwidth.Start, and changing speed to GEN3 data rate or higher in Loopback.Entry state.

When the above conditions are met, the SoC cannot detect TS OSs so that it fails to enter Loopback.Active state.

Impact:

A loopback test that meets the mentioned conditions will fail.

Workaround:

1. In a system design, avoid lane reversal if possible.
2. Else, the remote Loopback Master (a tester or a remote port) sets up loopback when starting training, by sending TS OSs with the Loopback bit set and the Compliance Receive bit set after LTSSM kicking off.
3. Else, the remote Loopback Master (a tester or a remote port) enters Loopback without link down, i.e., going through the Hot Reset state. This is expected to be the general Loopback Master's behavior.

Fix Status:

No SoC/HW/FW fix is needed. The test developers need to be aware of the limitation.

6.3. PCIe Controller changing data rate configuration from GEN4 to GEN3 or GEN2 has an issue

ID: Synopsys STAR ID 3916806 RTLMIN-6589

<https://esperantotech.atlassian.net/browse/RTLMIN-6589>

Owner: Daniel Yau

Block: PCIe Controller within PCIe Subsystem

Description:

The error was reported by Synopsys STAR report as shown in the ID section. When the controller is running at the GEN4 data rate, and either side of the link partners initiates a speed change to GEN3 or GEN2, the received 128b/130b EQ TS2s are not recognized.

As a result, the speed change failed.

Impact:

The speed change from GEN4 to GEN3 or to GEN2 does not work.

Workaround:

Make speed change from GEN4 to GEN1. Then, make another speed change from GEN1 to GEN3 or to GEN2.

Fix Status:

No SoC/HW fix is needed.

6.4. PCIe Power Management Functions are Unproven

ID: FV-330

<https://esperantotech.atlassian.net/browse/FV-330>

Owner: Daniel Yau

Block: PCIe Controllers within PCIe Subsystem

Description:

Some optional PCIe Power Management functions are excluded from the PCIe Controller design. The Power Management functions in the controllers are unproven. SW is advised to not use the Power Management functions.

Impact:

The Power Management functions are not used.

Workaround:

Disable the Power Management in SW.

Fix Status:

No SoC/HW fix is needed.

7. General

7.1. HPD PLL hold time violations

ID: FV-24 <https://esperantotech.atlassian.net/browse/FV-24>

Owner: Darren Jones

Block: Maxion Shire, IOShire/SPIO, Memshire, PShire

Description:

Since an incorrect timing model was used for the HPD PLL, there are hold time violations on the nine instances where the HPD PLL exists in the design. The nine instances are:

1. SPIO PLL0
2. SPIO PLL1
3. SPIO PLL2
4. SPIO PLL4
5. Maxshire pll_uncore
6. Maxshire pll_core
7. MemshireWestSide PLL
8. MemshireEastSide PLL
9. PShire PLL0

There are three key interfaces where hold time problems occur:

1. The TDR Interface. This is a test-only interface accessed through JTAG. The only violation that occurs is on a flop that is only used when bypassing TDR[0]. This violation occurs on all nine instances.
2. The APB Interface. This is the primary control interface used to program the PLL. Depending on the instance, the violations occur on control, address, and data signals. This violation only occurs on the following three instances:
 - a. Maxshire pll_uncore
 - b. Maxshire pll_core
 - c. MemshireWestSide PLL
3. The scan chain. This is a test-only interface used during ATE testing. Only MemshireEast has this violation, and it only exists at the FF corners.

The table below summarizes the hold problems on each PLL instance:

	TDR	APB	Scan
SPIO PLL0	X		
SPIO PLL1	X		

SPIO PLL2	X		
SPIO PLL4	X		
Maxshire Uncore	X	X	
Maxshire Core	X	X	
Memshire WestSide	X	X	
Memshire EastSide	X		X
PShire PLL0	X		

Impact:

The impact of the TDR violation is that data will be corrupted while bypassing the TDR[0] chain. The impact of the APB violation is that read and write accesses may be corrupted. The impact of the scan chain violation is that ATPG testing may not work reliably. Since ATE testing is done at room temperature, the impact could be minor.

Workaround:

The workaround for the TDR violation is to never bypass the TDR[0] chain. If you need to access the TDR[1] chain, then you must include the TDR[0] chain as well.

The workaround for the APB violation is to use the TDR interface instead of the APB interface to program the PLL. This workaround is acceptable in the lab, but not for production. Depending on characterization, we may be able to program the PLL at a lower voltage, and then ramp up the voltage during normal operation. If these workarounds are not used, this intermittent issue may occur. Software can detect this when programming the PLL by using the following procedure:

1. Write the PLL control registers.
2. Read back the PLL control registers to ensure proper values were written.
3. If the values are correct, write mem_pll_reg_update_strobe=1. This causes the values written in step (1) to become active.

If a workaround is necessary for the scan chain issue, fault coverage can be improved using additional functional testing. Running ATE tests at a lower voltage may also prevent the issue.

Fix Status:

Not Fixed

7.2. DFT scan chain router hold time violations

ID: FV-34 <https://esperantotech.atlassian.net/browse/FV-34>

Owner: Terry Beale

Block: Minion Shire

Description:

Due to a TDR set_case_analysis timing constraint, only the default DFT scan chain router path was timed in the Minion Shire, ioshire_misc, and ioshire_wrapper. This caused the other paths through the scan chain router to remain untimed.

The DFT team would like to run an ATPG scan on a selected Shire which requires all other Shires to have their scan chain router configured in bypass mode. Bypass mode just pipelines the scan chain data from the input of the Shire to the output on the opposite side. For the Minion Shire, the bypass path for the e2w direction meets hold timing, but the bypass path for the w2e direction has hold violations up to ~700 ps.

Impact:

The impact of the scan chain router hold violation is that the bypass path cannot be used in an optimal shift frequency configuration but can be configured to run with a bypass path that can run at a slower shift frequency and skip the hold violation.

Workaround:

The workaround for the hold violation is to bypass the first posedge flop in the w2e direction by setting the TDR value of dft__si_w2e_skip_pipe=1.

The bypass path normally receives the scan chain data from the previous Shire from a flop that is clocked on negedge. This goes into a three flop pipeline sequence of posedge-posedge-negedge flops. The hold violation exists between the first posedge and second posedge flop within the same Shire. By bypassing the first posedge flop, the bypass path launches from the previous Shire's negedge flops to the second posedge flop of the bypass path. This causes the Shire to have a two-flop pipeline sequence of posedge-negedge flops, skipping the hold violation between the two posedge flops within the same Shire.

The path from the previous Shire to the second posedge flop is longer and thus the scan shift speed must be slower to meet the setup time. However, this is not an issue because the scan shift speed is limited by other factors on the chip and this setup path meets that timing.

Fix Status:

Not fixed

7.3. HPDPLL and LVDPLL do not wait for LDO setup

ID: FV-120 <https://esperantotech.atlassian.net/browse/FV-120>

Owner: Miquel Izquierdo

Block: All the PLLs in the SoC. IP provided by Movellus

Description:

The Movellus PLLs include an LDO to provide a stable supply. The PLL initialization process includes different stages with some requirements. One of these requirements is that the DCO normalization process can only occur once the LDO is providing a stable supply. Hence, the initialization process must wait long enough for the LDO output to be stable before it starts the initialization of the DCO.

The PLL initialization state machine does not comply with the above requirement and it does not wait between the LDO initialization and DCO initialization process.

Impact:

The DCO normalization settles with the wrong values for the final DCO supply input. Consequently, the PLL is not able to achieve lock and a stable output clock. This is a major impact.

Workaround:

Workaround for the HPDPPLL:

While in BootROM, the LDO can be bypassed through the PLL configuration registers. The eFuses provide a mechanism to change BootROM behavior to enable the LDO bypass. See [ET SOC BootROM Debug-ability](#) for more details.

In Boot Loader 2, the PLL is configured again, enabling the LDO and forcing a reset on the LDO itself and the PLL. This process turns OFF the LDO and then back ON forcing it to restart its initialization process followed by a PLL disable and enable, to force the DCO normalization process to restart once the LDO supply is stable. See

[etsoc1_ioshire_clock_spec](#) for more details.

Workaround for the LVDPLL:

The expected workaround for the LVDPLL consists of a subset of the HPDPPLL workaround described in the following paragraph. However, this PLL is still not stable even with the workaround, and thus it seems likely that it is caused by a different issue.

When the PLL is initialized, a reset on the LDO itself and the PLL must be applied. This process turns OFF and then back ON the LDO, forcing it to restart its initialization process. This is followed by a PLL disable and enable to force the DCO normalization process to restart once the LDO supply is stable. See [etsoc1_ioshire_clock_spec](#) for more details.

Fix Status: Not Fixed.

8. ET-SOC A0 Erratum Template

8.1. <Template, enter here the title of the erratum>

ID: <Jira number> <link to Jira>

Owner: <who owns/edited/created this erratum>

Block: <a brief summary of where the bug is: e.g. Minion core, NBH, SP, Maxion, etc>

Description:

<detailed description:

Original intended proper/correct behaviour

Actual behaviour in A0 silicon>

Impact:

<Describe the severity of the issue (trivial/minor/major) and why it is given that level>

Workaround:

<Describe any software workarounds, if available>

Fix Status:

<Describe the plan to fix: in which version of silicon, if no plan, state “not fixed”>